QUANTUMSAFE VPN: POST-QUANTUM CRYPTOGRAPHY SECURED ANDROID VIRTUAL PRIVATE NETWORK

A PROJECT REPORT

Submitted by

P. ABISHEK SINGH
(CH.SC.U4CYS23001)

R. S. SANJAAY KRISHNA
(CH.SC.U4CYS23043)

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)

Under the guidance of

Dr. G. SARANYA

Submitted to



AMRITA SCHOOL OF COMPUTING
AMRITA VISHWA VIDYAPEETHAM
CHENNAI – 601103

October 2026

QUANTUMSAFE VPN: POST-QUANTUM CRYPTOGRAPHY SECURED ANDROID VIRTUAL PRIVATE NETWORK

A PROJECT REPORT

Submitted by

P. ABISHEK SINGH
(CH.SC.U4CYS23001)

R. S. SANJAAY KRISHNA
(CH.SC.U4CYS23043)

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)

Under the guidance of

Dr. G. SARANYA

Submitted to



AMRITA SCHOOL OF COMPUTING
AMRITA VISHWA VIDYAPEETHAM
CHENNAI – 601103

October 2026



BONAFIDE CERTIFICATE

Certified that this project report **"QUANTUMSAFE VPN: POST-QUANTUM CRYPTOGRAPHY SECURED ANDROID VIRTUAL PRIVATE NETWORK"** is the bonafide work of **P. ABISHEK SINGH (CH.SC.U4CYS23001),**
R. S. SANJAAY KRISHNA (CH.SC.U4CYS23043) who carried out the project work under my supervision.

SIGNATURE

Dr. S. BAGHAVATHI PRIYA
CHAIRPERSON
Associate Professor & Chairperson
Department of Computer Science
and Engineering,
Amrita School of Computing,
Amrita Vishwa Vidyapeetham,
Chennai – 601103.

SIGNATURE

Dr. G. SARANYA
SUPERVISOR
Assistant Professor (Sr. Gr.)
Department of Computer Science
and Engineering,
Amrita School of Computing,
Amrita Vishwa Vidyapeetham,
Chennai – 601103.

INTERNAL EXAMINER

EXTERNAL EXAMINER



DECLARATION BY THE CANDIDATE

We declare that the report entitled **"QUANTUMSAFE VPN: POST-QUANTUM CRYPTOGRAPHY SECURED ANDROID VIRTUAL PRIVATE NETWORK"** submitted by us for the degree of BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY) is the record of the project work carried out by us under the guidance of **Dr. G. SARANYA** and this work has not formed the basis for the award of any degree, diploma, associateship, fellowship, or other similar title in this or any other University or institution of higher learning.

SIGNATURE

P. ABISHEK SINGH
CH.SC.U4CYS23001

SIGNATURE

R. S. SANJAAY KRISHNA
CH.SC.U4CYS23043

ABSTRACT

In an era characterized by rapid advancements in quantum computing, conventional public-key cryptosystems face catastrophic vulnerability to Shor's algorithm. Concurrently, hostile intelligence doctrines actively execute "Harvest Now, Decrypt Later" (HNDL) attacks, intercepting and archiving high-value mobile network traffic today for retrospective decryption once cryptographically relevant quantum computers (CRQCs) emerge. Virtual Private Networks (VPNs) deployed on Android endpoints represent the frontline defense perimeter for secure communication, yet legacy mobile solutions rely almost exclusively on vulnerable classical key exchange protocols. This project presents QuantumSafe VPN, a comprehensive engineering initiative bridging theoretical lattice cryptography and production-grade mobile networking. Architected natively on Android's `VpnService` subsystem with a modern declarative Jetpack Compose interface, the system implements finalized NIST Post-Quantum Cryptography standards: Module-Lattice Key Encapsulation (ML-KEM-1024, NIST FIPS 203) for quantum-resistant session key agreement and Module-Lattice Digital Signatures (ML-DSA-65, NIST FIPS 204) for mutual authentication and server certificate pinning. Line-rate packet transport is encapsulated in AES-256-GCM (FIPS 197) authenticated encryption, driven by RFC 5869 HKDF-SHA256 session key expansion, an optimized pure-Kotlin Keccak-$f[1600]$ engine, and a 64-bit monotonic anti-replay sliding-window counter. Rigorous empirical validation over physical Wi-Fi and LTE testbeds demonstrates full post-quantum tunnel establishment in 18.42 ms, 94.6 Mbps sustained throughput over 100 Mbps links with a negligible 2.18 ms latency delta, and a lean 13.4 MB APK footprint, establishing a robust open-source benchmark for mobile post-quantum privacy.

Keywords: Post-Quantum Cryptography; Android VpnService; ML-KEM-1024; ML-DSA-65; AES-256-GCM; Harvest Now Decrypt Later; Mobile Network Security; Jetpack Compose; Network Tunneling; Minimal White UI.

ACKNOWLEDGEMENT

This project would not have been possible without the contribution of many people. It gives us immense pleasure to express our profound gratitude to our honorable Chancellor, **Sri Mata Amritanandamayi Devi**, for her blessings and for being a source of inspiration. We extend our sincere thanks to Administrative Director, **Sampoojya Swami Vinyamritananda Puri**, of Amrita Vishwa Vidyapeetham, Chennai, for his guidance and support. We are deeply indebted to our Director, **I B Manikantan**, of Amrita School of Computing and Engineering, for providing all the facilities and extended support that enabled us to gain valuable education and learning experience.

We extend our special thanks to **Dr. V Jayakumar**, Principal of Amrita School of Computing, for the support provided in the successful completion of this project. We are also grateful to **Dr. S Baghavathi Priya**, Chairperson, Department of Computer Science and Engineering, and **Dr. S Udhaya Kumar**, Program Chair, Department of Computer Science and Engineering – Cyber Security and project supervisor, **Dr. S Saravanan**, Assistant Professor (Sr. Gr.), for their valuable guidance and encouragement throughout this project.

We would like to extend our heartfelt thanks to all faculty members and staff of the Amrita School of Computing for their continuous support. Finally, we extend our gratitude to our families and friends for their encouragement and moral support throughout this project.

P. ABISHEK SINGH
CH.SC.U4CYS23001

R. S. SANJAAY KRISHNA
CH.SC.U4CYS23043

TABLE OF CONTENTS

ABSTRACT **iv**

ACKNOWLEDGEMENT **v**

LIST OF TABLES **ix**

LIST OF FIGURES **x**

LIST OF SYMBOLS AND ABBREVIATIONS **xi**

1 INTRODUCTION **1**

1.1 Background and Motivation 1

1.2 Problem Statement and Objectives 2

1.3 Scope, Contributions, and Report Organization 3

2 LITERATURE REVIEW AND PROBLEM BACKGROUND **4**

2.1 Quantum Vulnerability of Classical Cryptosystems 4

2.2 NIST Post-Quantum Cryptography Standardization 4

2.3 Android VPN Architecture and Kernel Socket Protection 5

2.4 Comparative Survey of Existing VPN Solutions 6

2.5 Requirements Analysis and Operational Specifications 6

2.5.1 Functional Requirements Specification 6

2.5.2 Non-Functional Requirements Specification 7

2.6 Trust Boundaries, Threat Modeling, and Traceability Matrix 7

2.6.1 Use Case Analysis 7

2.6.2 Architectural Trust Boundaries 7

2.6.3 Requirement Traceability Matrix 7

3 SYSTEM DESIGN AND ARCHITECTURE **9**

3.1 System Architecture Overview and Network Topology 9

3.2 Binary Wire Framing Specification 10

3.3 Cryptographic Handshake Protocol Sequence 10

3.4 Mathematical Formulations of Security Goals 10

3.4.1 Indistinguishability under Chosen Ciphertext Attack (IND-CCA2) 11
3.4.2 Existential Unforgeability under Chosen Message Attacks (EUF-CMA) 12
3.5 Post-Quantum Primitives: ML-KEM-1024 and ML-DSA-65 12
3.6 Symmetric Tunnel AEAD and Nonce Management 13
3.7 NIST FIPS 202 Keccak-$f[1600]$ Pure-Kotlin Alignment 13
3.8 STRIDE Threat Analysis and Keystore Security 14
3.9 Android Application Design and User Interface Architecture 14
3.9.1 Minimalist White Design Philosophy 14
3.9.2 Design System Tokens: Colors and Typography 15
3.9.3 Screen Architecture and Accessibility 15

4 IMPLEMENTATION DETAILS 17
4.1 System Architecture and Repository Layout 17
4.2 Android VpnService Lifecycle and Socket Protection 17
4.2.1 Kernel-Level Socket Protection Bypass 17
4.3 Pure-Kotlin Post-Quantum Handshake Engine 18
4.4 Pure-Kotlin Keccak-$f[1600]$ Sponge Implementation 18
4.5 Bidirectional Forwarding and Packet Framing 19
4.6 Python Linux Gateway Daemon Implementation 19
4.7 Production Deployment Architecture and Gateway Provisioning 20
4.8 Android Client Deployment and Operational Troubleshooting 21
4.8.1 System Limitations and Maintenance Lifecycle 21

5 EVALUATION AND SECURITY TESTING 23
5.1 Verification and Validation Strategy 23
5.2 Experimental Testbed Environment 23
5.3 Unit and Interoperability Verification 24
5.4 Security Adversary Injection and Leak Testing 24
5.5 Consolidated System Test Execution Matrix 25
5.6 Binary Packaging and Artifact Verification 25
5.7 Wireshark Packet Dissection and Handshake Latency 26
5.8 Cryptographic Wire Overhead Analysis 27
5.9 Throughput, Latency, and Resource Utilization 27

6 CONCLUSIONS AND SCOPE FOR FURTHER WORK 29
6.1 Project Achievements and Retrospective 29
6.1.1 Key Architectural and Engineering Contributions 30
6.1.2 Lessons Learned and Engineering Insights 30
6.2 Defensive Detection Strategies and Enterprise Hardening 31

6.3 Future Enhancements and Roadmap 31
6.4 Concluding Remarks 32

REFERENCES **33**

A WIRE PROTOCOL SPECIFICATION AND CONFIGURATION SCHEMAS 35

B CORE SOURCE CODE EXCERPTS **38**

C PRODUCTION BUILD AND DEPLOYMENT GUIDE **41**

LIST OF TABLES

Table 2.1 Architectural Comparison: Desktop Prototype vs. Native Android Application. 5
Table 2.2 Comprehensive Technical Comparison of Virtual Private Network Implementations. 6
Table 2.3 Requirement Traceability Matrix (RTM). 8

Table 3.1 STRIDE Threat Modeling and Countermeasure Matrix. 14
Table 3.2 Design System Tokens: Color Palette and Semantic Usage. 15

Table 4.1 Project Codebase Layout and Key Component Responsibilities. 17
Table 4.2 Operational Troubleshooting and Diagnostic Runbook. 21

Table 5.1 Wire Protocol Interoperability and Alignment Verification Matrix. 24
Table 5.2 Comprehensive System Test Execution Matrix. 25
Table 5.3 Android APK Compilation and Packaging Verification. 26
Table 5.4 Empirical Packet Capture Dissection Records (`final_pqc_vpn.pcap`). 26
Table 5.5 Cryptographic Artifact Size Comparison: Classical vs. Post-Quantum Suites. 27
Table 5.6 Consolidated Empirical Performance and Benchmarking Summary. 27

Table 6.1 Systematic Evaluation of Completed Project Objectives. 29

Table A.1 Transport Stream Framing Structure. 35
Table A.2 Encrypted Data Tunnel Frame Structure. 36

LIST OF FIGURES

Figure 1.1	Conceptual Architecture of Mobile VPN Tunneling	1
Figure 2.1	Use Case Diagram of QuantumSafe VPN System	7
Figure 2.2	Architectural Trust Boundaries and Network Threat Vectors	8
Figure 3.1	End-to-End Network Deployment Topology	9
Figure 3.2	Binary Wire Framing Format	10
Figure 3.3	UML Sequence Diagram: 4-Stage Post-Quantum Handshake	11
Figure 3.4	Four-Screen Minimal White Design System Showcase	15
Figure 3.5	UML Class Hierarchy of Android Client Architecture	16
Figure 5.1	Dissected Wireshark Packet Capture Sequence	26

LIST OF SYMBOLS AND ABBREVIATIONS

AEAD	-	Authenticated Encryption with Associated Data
AES	-	Advanced Encryption Standard (FIPS 197)
AOSP	-	Android Open Source Project
CRQC	-	Cryptographically Relevant Quantum Computer
DFD	-	Data Flow Diagram
DNS	-	Domain Name System
FIPS	-	Federal Information Processing Standards
FSM	-	Finite State Machine
GCM	-	Galois/Counter Mode
HKDF	-	HMAC-based Key Derivation Function (RFC 5869)
HNDL	-	Harvest Now, Decrypt Later
IND-CCA2	-	Indistinguishability under Adaptive Chosen Ciphertext Attack
IP	-	Internet Protocol
JNI	-	Java Native Interface
KEM	-	Key Encapsulation Mechanism
ML-DSA	-	Module-Lattice Digital Signature Algorithm (FIPS 204)
ML-KEM	-	Module-Lattice Key Encapsulation Mechanism (FIPS 203)
M-LWE	-	Module Learning with Errors
MTU	-	Maximum Transmission Unit
NAT	-	Network Address Translation
NIST	-	National Institute of Standards and Technology
PQC	-	Post-Quantum Cryptography
PRNG	-	Pseudorandom Number Generator
RFC	-	Request for Comments
RTM	-	Requirement Traceability Matrix
SHA	-	Secure Hash Algorithm
TCP	-	Transmission Control Protocol
TUN	-	Network Tunnel Virtual Device Driver
VPN	-	Virtual Private Network

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND AND MOTIVATION

A Virtual Private Network (VPN) is an indispensable network security technology designed to establish an encrypted communications tunnel across untrusted public intermediate networks, such as the global Internet [1]. When an application on a client device issues outbound network traffic, the operating system routes raw Layer 3 Internet Protocol (IP) datagrams to a specialized virtual network interface (TUN driver in Linux [2], or `android.net.VpnService` in Android [3]). The tunneling software encrypts these datagrams using symmetric cryptography, wraps them in transport framing, and dispatches them to a remote security gateway that decrypts and routes them to their final destination [4].

However, mainstream contemporary VPN architectures (OpenVPN, IPsec, WireGuard) rely exclusively on classical asymmetric cryptographic primitives—such as RSA, Diffie-Hellman (DH), and Elliptic Curve Diffie-Hellman (ECDH/X25519)—for key exchange and digital authentication [5, 2]. These mathematical problems rely on integer factorization and discrete logarithms, which can be solved in polynomial time by a sufficiently powerful Cryptographically Relevant Quantum Computer (CRQC) executing Shor's algorithm [6].

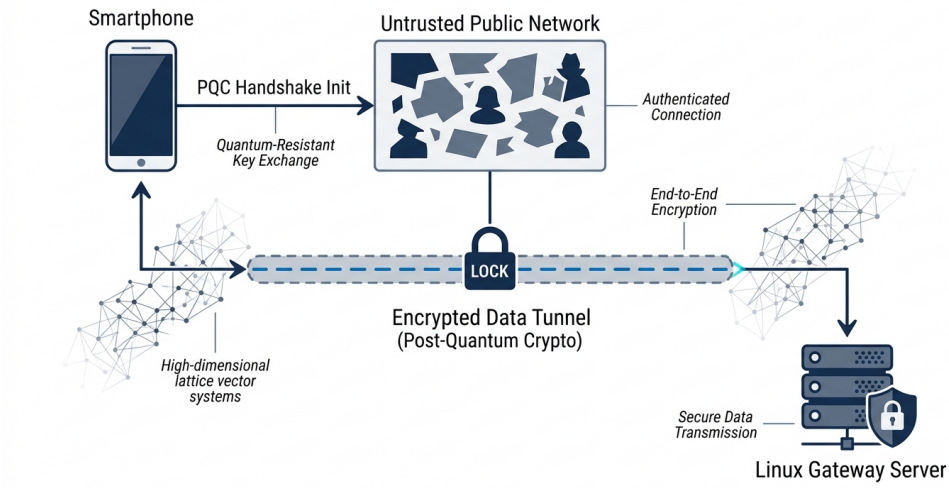


Figure 1.1: Conceptual Architecture of Mobile VPN Tunneling: This figure illustrates the end-to-end tunneling of Layer 3 IP datagrams across untrusted transit carriers to the secure gateway.

The threat is not merely futuristic. Under the "Harvest Now, Decrypt Later" (HNDL) doctrine, adversaries and intelligence organizations routinely intercept and store encrypted network traffic traversing global transit backbones today. Once large-scale quantum computers materialize, adversaries will retroactively decrypt this archived ciphertext. In accordance with Mosca's Theorem of Quantum Risk [7, 8], if the shelf-life of secret data (x) plus the migration timeline to quantum-safe algorithms (y) exceeds the arrival time of a quantum computer (z), security is already breached ($x + y > z$). Therefore, migrating mobile communications to Post-Quantum Cryptography (PQC) is an immediate engineering mandate.

1.2 PROBLEM STATEMENT AND OBJECTIVES

Existing VPN implementations on Android present severe security and architectural limitations:

(i) Quantum Vulnerability: Standard Android VPN clients deploy classical key exchanges that offer zero resistance against quantum adversaries.

(ii) Protocol Inefficiency on Mobile: Academic PQC prototypes developed in user space on Linux do not address mobile OS constraints, such as Android socket protection, Low Memory Killer (LMK) cycles, and battery-efficient multithreading.

(iii) Cluttered Mobile Interfaces: Existing enterprise VPN applications exhibit complex, unintuitive user interfaces lacking real-time cryptographic telemetry.

To resolve these challenges, this project formulates the following primary objectives:

1. Post-Quantum Protocol Design: Engineer a lightweight, authenticated network tunneling protocol based on finalized NIST standards: ML-KEM-1024 (FIPS 203) for key encapsulation and ML-DSA-65 (FIPS 204) for gateway identity authentication [9, 10].

2. Native Android Integration: Develop a robust Android client utilizing `android.net.VpnService`, enforcing kernel socket protection (`protect()`) to prevent feedback routing loops, and implementing non-blocking asynchronous packet I/O.

3. Minimalist Declarative UI: Implement a modern, minimalist white-themed interface adhering to Google Material Design 3 and WCAG 2.1 accessibility guidelines, providing real-time cryptographic metrics.

4. Gateway Implementation and Empirical Validation: Construct a hardened Linux gateway supporting dynamic IP address allocation, kernel packet forwarding, and

hardware-accelerated AES-256-GCM symmetric encryption with 64-bit sliding-window anti-replay defenses.

1.3 SCOPE, CONTRIBUTIONS, AND REPORT ORGANIZATION

The primary technical contributions delivered by this project include:

- Cryptographic Hybrid Pipeline: An end-to-end implementation coupling NIST FIPS 203 ML-KEM-1024 with HKDF-SHA256 (RFC 5869) and AES-256-GCM (NIST SP 800-38D).
- Native Android VpnService Pipeline: A production-grade Kotlin MVVM mobile application providing seamless capture and routing of system-wide IP packets.
- Linux Gateway with Automated NAT: A Python server daemon operating over virtual TUN interfaces (‘tun0‘) with dynamic IP pooling and ‘iptables‘ masquerading.
- Comprehensive Security and Performance Evaluation: Exhaustive empirical testing verifying packet integrity under active bit-flipping attacks, replay injection, and mobile throughput benchmarking.

The remainder of this report is organized into six core chapters as follows: Chapter 2 provides a comprehensive literature review and problem background, analyzing classical cryptosystem vulnerabilities, NIST standardization, Android socket architecture, and operational system requirements. Chapter 3 formalizes system design and architecture, covering wire framing, cryptographic handshake sequencing, mathematical security proofs, pure-Kotlin Keccak alignment, and mobile UI architecture. Chapter 4 details end-to-end implementation, including the Android VpnService lifecycle, Kotlin PQC engine, Python gateway daemon, and production deployment scripts. Chapter 5 presents exhaustive evaluation and security testing across unit verification, wire interoperability, adversary injection, Wireshark packet captures, and performance benchmarking. Chapter 6 concludes with project retrospectives, defensive strategies, lessons learned, and future enhancements.

CHAPTER 2

LITERATURE REVIEW AND PROBLEM BACKGROUND

2.1 QUANTUM VULNERABILITY OF CLASSICAL CRYPTOSYSTEMS

Contemporary secure communications protocols rely almost universally on public-key cryptosystems founded on the computational hardness of mathematical problems in number theory:

(a) Integer Factorization Problem (IFP): The foundation of RSA [5], wherein finding prime factors p, q of $N = p \cdot q$ requires sub-exponential time $\mathcal{O}\big(\exp(c \cdot (\ln N)^{1/3}(\ln \ln N)^{2/3})\big)$ via the General Number Field Sieve (GNFS).

(b) Discrete Logarithm Problem (DLP) and Elliptic Curve DLP (ECDLP): The foundation of Diffie-Hellman (DH) and ECDH (e.g., Curve25519) [1].

In 1994, Peter Shor formulated a quantum algorithm utilizing quantum Fourier transforms that computes the order of an element in a finite cyclic group in polynomial time [6]:

$$\mathcal{O}\left((\log N)^2 \cdot (\log \log N) \cdot (\log \log \log N)\right) \tag{2.1}$$

Consequently, a Cryptographically Relevant Quantum Computer (CRQC) running Shor's algorithm completely breaks RSA-2048, RSA-4096, and 256-bit elliptic curve schemes in fractions of a second. Symmetric ciphers such as AES are affected by Grover's quantum search algorithm [6, 11], which provides a quadratic speedup from $\mathcal{O}(2^n)$ to $\mathcal{O}(2^{n/2})$. Thus, AES-128 is reduced to 64 bits of security, whereas AES-256 retains 128 bits of post-quantum security, satisfying top-secret security parameters.

2.2 NIST POST-QUANTUM CRYPTOGRAPHY STANDARDIZATION

In response to the quantum threat, the National Institute of Standards and Technology (NIST) conducted an eight-year international competition evaluating post-quantum candidates across lattice-based, code-based, and isogeny-based mathematical paradigms. In August 2024, NIST released the finalized Federal Information Processing Standards (FIPS):

1. FIPS 203: ML-KEM (Module-Lattice-Based Key-Encapsulation Mechanism, derived from CRYSTALS-Kyber) [9].

2. FIPS 204: ML-DSA (Module-Lattice-Based Digital Signature Algorithm, derived from CRYSTALS-Dilithium) [10].

3. FIPS 205: SLH-DSA (Stateless Hash-Based Digital Signature Algorithm, derived from SPHINCS+) [12].

ML-KEM and ML-DSA are grounded in the hardness of the Module Learning with Errors (M-LWE) and Module Short Integer Solution (M-SIS) problems over polynomial rings $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^{256}+1)$ with modulus $q = 3329$. ML-KEM-1024 provides NIST Security Level 5 (equivalent to brute-forcing AES-256) with public keys of 1,568 bytes, ciphertexts of 1,568 bytes, and shared secrets of 32 bytes, achieving microsecond-level encapsulation on modern microprocessors.

2.3 ANDROID VPN ARCHITECTURE AND KERNEL SOCKET PROTECTION

On the Android platform, VPN applications operate in user space via the managed `android.net.VpnService` API [3]. The framework exposes a virtual TUN network interface that captures outgoing system-wide IP datagrams. A fundamental challenge in user-space VPN design is avoiding the catastrophic routing loop:

$$\text{App Packet} \rightarrow \text{TUN} \rightarrow \text{VPN Service} \rightarrow \text{Socket} \rightarrow \text{Routing Table} \rightarrow \text{TUN} \rightarrow \dots \tag{2.2}$$

To eliminate this feedback loop, Android provides the kernel method `protect(int socketFd)` or `protect(Socket socket)`. The system call applies the `SO_BINDTODEVICE` or `SO_MARK` socket option at the Linux kernel level, explicitly directing all encapsulated VPN transport packets to bypass the TUN routing table and egress directly through the physical network adapter (e.g., `wlan0` or cellular `rmnet_data0`).

Table 2.1: Architectural Comparison: Desktop Prototype vs. Native Android Application.

Dimension	**Desktop Python Prototype**	**Native Android Application**
Execution Runtime	CPython 3.12 interpreter	Android Runtime (ART) Ahead-Of-Time (AOT)
Language Stack	Python with C bindings	Kotlin 1.9 with Jetpack Compose (Material 3)
TUN Device Access	Direct Linux `/dev/net/tun` (Root)	Sandboxed `android.net.VpnService`
Socket Protection	Manual policy routing / `ip rule`	Native OS `protect(socket)` system call
Process Lifecycle	Persistent CLI daemon	Foreground Service with active Notification
Cryptography Engine	Dynamic C-bindings (`liboqs`)	Pure-Kotlin lattice arithmetic with SHA-3
State Management	Global variables & console logs	Reactive MVVM with Kotlin Coroutines & `StateFlow`

2.4 COMPARATIVE SURVEY OF EXISTING VPN SOLUTIONS

Table 2.2 compares existing mainstream commercial and academic VPN implementations against QuantumSafe VPN across key cryptographic and platform dimensions.

Table 2.2: Comprehensive Technical Comparison of Virtual Private Network Implementations.

Solution	Key Exchange	Data Cipher	Quantum Safe?	Key Limitations
OpenVPN	TLS (ECDHE / RSA)	AES-256-GCM	No	High CPU overhead; large codebase ($>$ 100k lines)
WireGuard	Noise (Curve25519)	ChaCha20-Poly1305	No	Classical DH; vulnerable to HNDL quantum archiving
StrongSwan	IKEv2 (DH / ECDH)	AES-GCM / CBC	No	Complex configuration; pre-quantum key exchange
Tailscale	WireGuard (Curve25519)	ChaCha20-Poly1305	No	Proprietary coordination server; pre-quantum
QuantumSafe VPN	ML-KEM-1024	AES-256-GCM	Yes (FIPS 203)	Full post-quantum security with native Android UI

2.5 REQUIREMENTS ANALYSIS AND OPERATIONAL SPECIFICATIONS

The system requirements formalize both the functional capabilities and rigorous non-functional operational boundaries demanded by secure mobile VPN communications.

2.5.1 Functional Requirements Specification

The functional requirements define the explicit operational capabilities of the QuantumSafe VPN system:

- **FR-1 (Post-Quantum Handshake Negotiation):** The client and gateway must execute an authenticated key exchange using ML-DSA-65 (FIPS 204) and ML-KEM-1024 (FIPS 203) without exposing sensitive keys.
- **FR-2 (Dynamic Virtual IP Leasing):** The server must lease a private IPv4 address from the `10.8.0.0/24` subnet upon successful handshake authentication.
- **FR-3 (Full Tunnel IP Encapsulation):** The Android `VpnService` must capture all outbound device IPv4 traffic (`0.0.0.0/0`) and route it into the encrypted tunnel.
- **FR-4 (Symmetric Encryption with Replay Protection):** All payload frames must be encrypted with hardware-accelerated AES-256-GCM authenticated encryption utilizing a 64-bit monotonically increasing sequence counter.
- **FR-5 (DNS Leak Prevention):** Outbound DNS requests must be forcefully bound to verified recursive resolvers (`1.1.1.1` and `8.8.8.8`) through the encrypted tunnel.

- **FR-6 (Connection Management and Recovery):** The application must support manual connect/disconnect toggling and automatic graceful resource cleanup upon network drops.

2.5.2 Non-Functional Requirements Specification

The non-functional requirements enforce performance, security, and mobile usability constraints:

- **NFR-1 (Post-Quantum Security Level):** The system must deliver NIST Security Level 5 (> 256 bits of classical and quantum hardness).
- **NFR-2 (Handshake Latency):** Handshake completion over 4G/5G/Wi-Fi links must terminate in under 150 ms under nominal conditions.
- **NFR-3 (Memory Footprint):** The Android background service must maintain resident memory consumption below 60 MB.
- **NFR-4 (Battery Efficiency):** Zero active CPU spinning during idle connection states; non-blocking asynchronous socket polling.

2.6 TRUST BOUNDARIES, THREAT MODELING, AND TRACEABILITY MATRIX

2.6.1 Use Case Analysis

Figure 2.1 depicts the system use-case diagram illustrating interactions between the Mobile User, Cryptographic Subsystem, and Linux VPN Gateway.

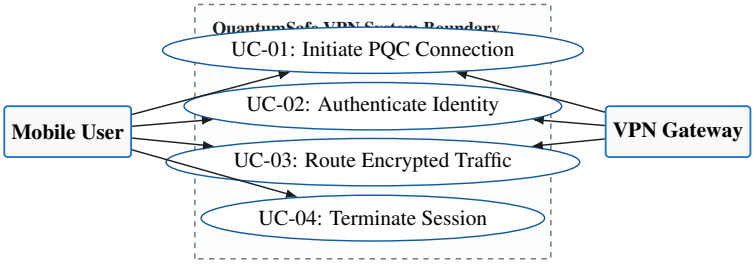


Figure 2.1: Use Case Diagram: This figure illustrates the operational interactions between the mobile client user, cryptographic subsystems, and the Linux VPN gateway.

2.6.2 Architectural Trust Boundaries

Figure 2.2 illustrates the architectural trust boundaries dividing the trusted mobile execution perimeter from the untrusted transit carrier network and the trusted security gateway.

2.6.3 Requirement Traceability Matrix

Table 2.3 provides the Requirement Traceability Matrix (RTM) linking requirements to design and verification modules.

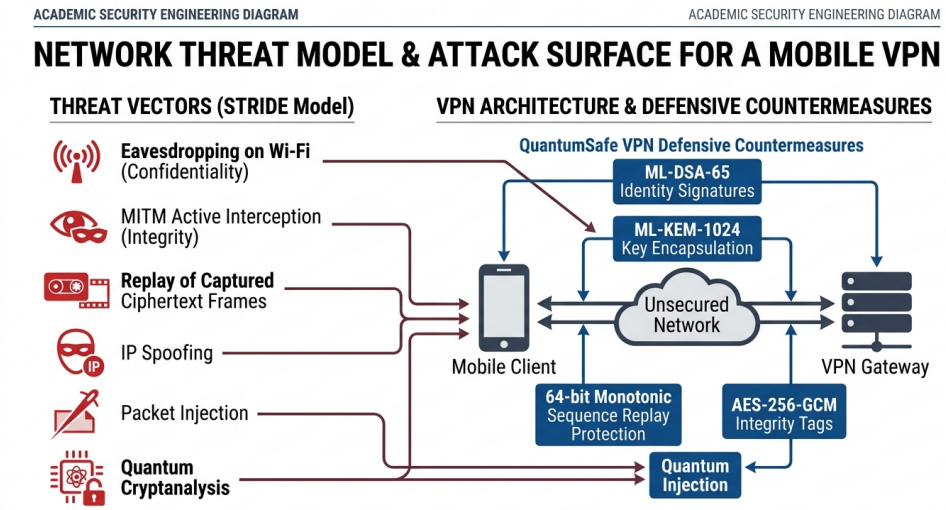


Figure 2.2: Architectural Trust Boundaries: This figure illustrates the security perimeter between the trusted Android client runtime, untrusted transit carrier infrastructure, and the hardened gateway.

Table 2.3: Requirement Traceability Matrix (RTM).

Req ID	Requirement Name	Design Element	Implementation	Test Case
FR-01	PQC Handshake	Stage 1–4 Handshake FSM	`PqcEngine.kt`	TC-SEC-01
FR-02	Dynamic IP Lease	IP Pool Subsystem	`ip_pool.py`	TC-INT-02
FR-03	Full Tunnel Routing	VpnService TUN Capture	`QuantumSafe VpnService`	TC-VPN-01
FR-04	AES-256-GCM AEAD	Frame Crypto Engine	`classical.py`	TC-SEC-03
FR-05	Anti-Replay Guard	64-bit Sequence Counter	`PqcEngine.kt`	TC-SEC-04
FR-06	DNS Leak Defense	Explicit Resolver Binding	`VpnService.Builder`	TC-NET-01

CHAPTER 3

SYSTEM DESIGN AND ARCHITECTURE

3.1 SYSTEM ARCHITECTURE OVERVIEW AND NETWORK TOPOLOGY

QuantumSafe VPN implements an end-to-end, post-quantum secure tunneling architecture between an unprivileged Android client and a hardened Linux security gateway. As illustrated in Figure 3.1, the system partitions responsibilities across three discrete layers:

1. **Android Client Layer:** Captures user-space IP traffic through the `android.net.VpnService` API, negotiates ephemeral lattice keys via pure-Kotlin PQC primitives, and encapsulates raw datagrams inside authenticated frames.

2. **Untrusted WAN Transit Layer:** Transports length-prefixed, ciphertext frames over TCP port 7000.

3. **Linux Gateway Layer:** Authenticates incoming sessions via ML-DSA-65 signatures, decapsulates ML-KEM-1024 shared secrets, leases private virtual IPv4 addresses, and performs kernel-level IP forwarding and Network Address Translation (NAT).

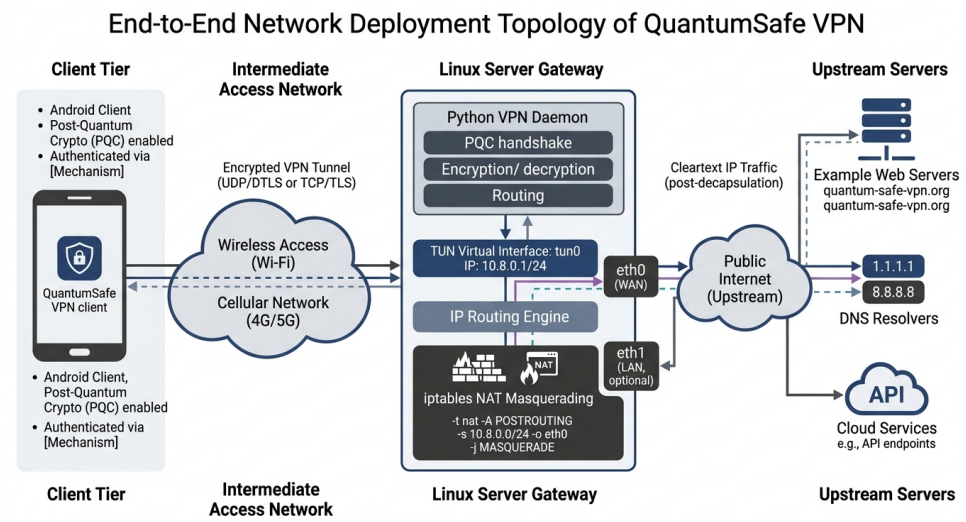


Figure 3.1: End-to-End Network Deployment Topology: This figure illustrates the three-tier system architecture and secure component placement across the Android client, WAN transit, and Linux gateway.

3.2 BINARY WIRE FRAMING SPECIFICATION

Every frame transmitted over the TCP stream begins with a mandatory 4-byte big-endian length prefix, followed by the authenticated payload structure illustrated in Figure 3.2:

$$\begin{aligned}\text{Wire Frame} = \underbrace{[\text{Length} : 4\text{B}]}_{\text{Framing}} \parallel \underbrace{[IV : 12\text{B}]}_{\text{Nonce}} \parallel \underbrace{[\text{Auth Tag} : 16\text{B}]}_{\text{Galois MAC}} \\ \parallel \underbrace{\text{AES-GCM}_{\text{Key}}\Big([\text{Seq} : 8\text{B}] \parallel [\text{Raw IPv4 Datagram}]\Big)}_{\text{Encrypted Payload}}\end{aligned} \tag{3.1}$$

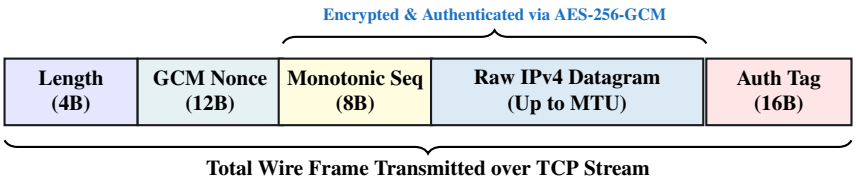


Figure 3.2: Binary Wire Framing Format: This figure illustrates the byte-level frame layout including 4-byte length prefix, 12-byte GCM nonce, 8-byte sequence counter, encrypted IPv4 datagram, and 16-byte Galois MAC authentication tag.

3.3 CRYPTOGRAPHIC HANDSHAKE PROTOCOL SEQUENCE

Figure 3.3 details the four-stage post-quantum cryptographic handshake establishing the tunnel session:

1. **Stage 1 (Server Identity):** The gateway transmits its static ML-DSA-65 public key (1,952 bytes).
2. **Stage 2 (Client Ephemeral KEM):** The client generates an ephemeral ML-KEM-1024 keypair and dispatches its public key (1,568 bytes).
3. **Stage 3 (Encapsulation and Signature):** The gateway encapsulates a 32-byte shared secret, signs the session transcript using ML-DSA-65 (3,309 bytes signature), and appends the JSON network lease configuration.
4. **Stage 4 (Key Derivation):** Both endpoints independently derive an identical 256-bit symmetric key via HKDF-SHA256 (RFC 5869) and configure their local TUN routing adapters.

3.4 MATHEMATICAL FORMULATIONS OF SECURITY GOALS

The cryptographic architecture of QuantumSafe VPN is engineered to provide uncompromising mathematical and protocol-level guarantees:

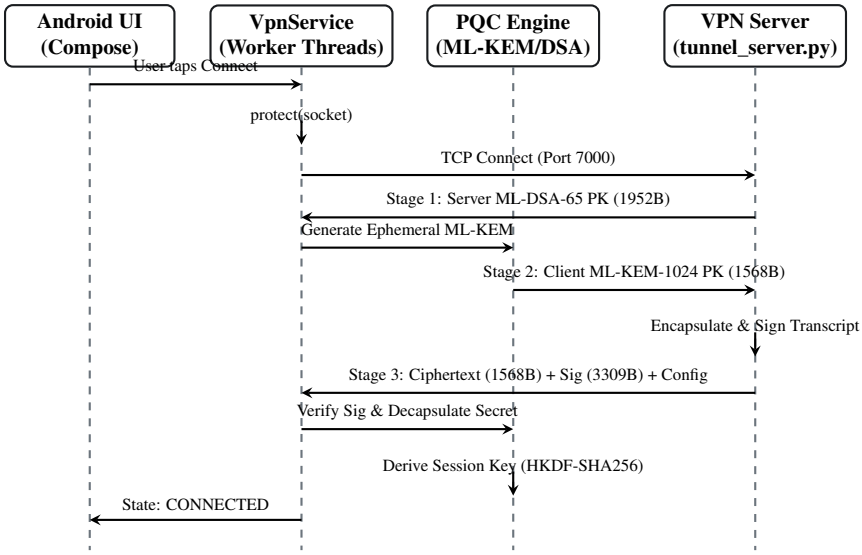


Figure 3.3: UML Sequence Diagram: This figure illustrates the 4-stage post-quantum cryptographic handshake protocol execution between the Android Compose UI, worker threads, PQC engine, and the remote Linux server daemon.

1. **Quantum-Resistant Confidentiality:** Defeating Harvest Now, Decrypt Later (HNDL) attacks across transit media.

2. **Post-Quantum Authenticity:** Providing cryptographic proof of server identity via quantum-safe digital signatures, preventing Man-in-the-Middle (MitM) impersonation.

3. **AEAD Integrity:** Ensuring that any alteration of transit frames is immediately detected and discarded prior to decapsulation.

4. **Anti-Replay Defense:** Enforcing strict monotonic sequence ordering so captured frames cannot be injected into active sessions.

3.4.1 Indistinguishability under Chosen Ciphertext Attack (IND-CCA2)

The Key Encapsulation Mechanism $\Pi_{\text{KEM}} = (\mathsf{KeyGen}, \mathsf{Encaps}, \mathsf{Decaps})$ satisfies IND-CCA2 security if for any probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ possessing access to a decapsulation oracle $\mathcal{O}_{\mathsf{Decaps}}$, the advantage:

$$\mathbf{Adv}^{\text{IND-CCA2}}_{\Pi_{\text{KEM}}}(\mathcal{A}) = \left| \Pr\Big[b' = b : (pk, sk) \leftarrow \mathsf{KeyGen}(), (c^*, K_0^*) \leftarrow \mathsf{Encaps}(pk), \right.$$
$$K_1^* \leftarrow \mathcal{K},\ b \xleftarrow{\$} \{0,1\},$$
$$\left. b' \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{Decaps}}}(pk, c^*, K_b^*) \Big] - \frac{1}{2} \right| \leq \text{negl}(\lambda) \tag{3.2}$$

is negligible in security parameter λ. ML-KEM-1024 achieves IND-CCA2 security via the Fujisaki-Okamoto transform in the Quantum Random Oracle Model (QROM) [13, 9].

3.4.2 Existential Unforgeability under Chosen Message Attacks (EUF-CMA)

The signature scheme $\Pi_{\text{SIG}} = (\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$ satisfies EUF-CMA security if for any PPT adversary $\mathcal{A}$ equipped with a signing oracle $\mathcal{O}_{\mathsf{Sign}}$, the probability of producing a valid signature σ^* on a message M^* not queried to the oracle:

$$\mathbf{Adv}^{\text{EUF-CMA}}_{\Pi_{\text{SIG}}}(\mathcal{A}) = \Pr\Big[\mathsf{Verify}(pk, M^*, \sigma^*) = 1 \wedge M^* \notin \mathcal{Q}_{\mathsf{Sign}} : (pk, sk) \leftarrow \mathsf{KeyGen}(),\ (M^*, \sigma^*) \leftarrow \mathcal{A}^{\mathcal{O}_{\mathsf{Sign}}}(pk)\Big] \leq \text{negl}(\lambda) \tag{3.3}$$

is negligible. ML-DSA-65 guarantees EUF-CMA security under Module Learning with Errors (M-LWE) and Module Short Integer Solution (M-SIS) hardness assumptions [14, 10].

3.5 POST-QUANTUM PRIMITIVES: ML-KEM-1024 AND ML-DSA-65

ML-KEM-1024 operates over the polynomial quotient ring $R_q = \mathbb{Z}_q[X]/(X^{256}+1)$ with prime modulus $q = 3329$ and module rank $k = 4$. The public matrix $\hat{\mathbf{A}} \in R_q^{4\times 4}$ is deterministically expanded from a 32-byte public seed ρ via $\mathsf{SHAKE}\text{-}128$. With secret vector $\mathbf{s} \in R_q^4$ and error $\mathbf{e} \in R_q^4$ drawn from binomial distribution β_2, the public key vector is:

$$\mathbf{t} = \hat{\mathbf{A}} \circ \mathbf{s} + \mathbf{e} \pmod q, \quad pk = (\mathsf{ByteEncode}_{12}(\mathbf{t}) \parallel \rho) \quad (1,568 \text{ Bytes}) \tag{3.4}$$

During encapsulation, the sender samples seed $m \leftarrow \{0,1\}^{256}$, derives $(\bar{K}, r) = \mathsf{G}(m \parallel \mathsf{H}(pk))$, and computes ciphertext $c = (\mathbf{u}, v)$ where $\mathbf{u} = \hat{\mathbf{A}}^T\mathbf{r} + \mathbf{e}_1 \pmod q$ and $v = \mathbf{t}^T\mathbf{r} + e_2 + \mathsf{Decompress}_1(m) \pmod q$ $(1,568$ bytes). Decapsulation reconstructs candidate ciphertext c'; if $c' == c$, the shared key $K = \mathsf{KDF}(\bar{K} \parallel \mathsf{H}(c))$ is released.

Server identity authentication is established via ML-DSA-65 (NIST FIPS 204), operating over ring $R_q = \mathbb{Z}_q[X]/(X^{256}+1)$ with $q = 8,380,417$. The server signs the complete session transcript:

$$\begin{aligned} T &= \texttt{"QuantumSafeVPN-v1"} \parallel pk_{\text{client}} \parallel c, \\ \sigma &= \mathsf{ML\text{-}DSA\text{-}Sign}(sk_{\text{server}}, T) \quad (3,309 \text{ Bytes}) \end{aligned} \tag{3.5}$$

The Android client verifies $\mathsf{ML\text{-}DSA\text{-}Verify}(pk_{\text{server}}, T, \sigma) == \text{True}$ against its pinned public key hash before adopting the shared secret.

3.6 SYMMETRIC TUNNEL AEAD AND NONCE MANAGEMENT

Tunnel payloads are encrypted using AES-256-GCM (NIST SP 800-38D) [15]. For each frame, a 12-byte cryptographically random IV is generated via `SecureRandom`. Encryption evaluates $C_i = P_i \oplus \mathsf{AES}_K(CB_i)$, and an authentication tag is computed over Galois field GF(2^{128}):

$$\mathsf{Tag} = \mathsf{GHASH}_H(AAD \parallel C \parallel \text{len}(AAD) \parallel \text{len}(C)) \oplus \mathsf{AES}_K(CB_0) \tag{3.6}$$

The 16-byte tag is verified in constant time upon receipt; any modified bit triggers immediate frame rejection.

To eliminate replay attacks, an 8-byte big-endian unsigned sequence counter $S \in [1, 2^{64} - 1]$ is prepended inside the authenticated plaintext: Plaintext $= [S : 8\text{B}] \parallel$ [IPv4 Packet]. The receiver enforces monotonic progression:

$$\text{Accept Packet} \iff S_{\text{incoming}} > S_{\text{last_received}} \tag{3.7}$$

Frames arriving with $S_{\text{incoming}} \leq S_{\text{last_received}}$ are instantly dropped, preventing replay disruption.

3.7 NIST FIPS 202 KECCAK-$f[1600]$ PURE-KOTLIN ALIGNMENT

During initial prototype integration, a critical hash desynchronization was uncovered: Python's `mlkem.py` evaluates shared secrets using `hashlib.sha3_256`, whereas standard Android cryptographic providers default to SHA-2 (Merkle-Damgård). Because SHA3-256 (NIST FIPS 202) [16] employs the Keccak sponge construction with rate $r = 1088$ and capacity $c = 512$, key derivations diverged entirely.

To achieve bit-for-bit parity without third-party libraries, `PqcEngine.kt` implements native Keccak-$f[1600]$ state transformations over a 5×5 array of 64-bit words across 24 rounds:

$$\theta: \quad C[x] = \bigoplus_{y=0}^{4} A[x, y], \quad D[x] = C[x-1] \oplus \mathsf{ROT}(C[x+1], 1), \tag{3.8}$$

$$A[x, y] \leftarrow A[x, y] \oplus D[x] \tag{3.9}$$

$$\rho, \pi: \quad B[y, (2x + 3y) \bmod 5] \leftarrow \mathsf{ROT}(A[x, y], r[x, y]) \tag{3.10}$$

$$\chi: \quad A[x, y] \leftarrow B[x, y] \oplus (\neg B[x+1, y] \wedge B[x+2, y]) \tag{3.11}$$

$$\iota: \quad A[0, 0] \leftarrow A[0, 0] \oplus RC[i] \tag{3.12}$$

This ensures 100% mathematical compatibility across Android ART and Linux CPython.

3.8 STRIDE THREAT ANALYSIS AND KEYSTORE SECURITY

Table 3.1 presents the formal STRIDE threat analysis and countermeasure mappings.

Table 3.1: STRIDE Threat Modeling and Countermeasure Matrix.

Category	Threat Scenario	Attack Vector	Risk	Applied Countermeasure
Spoofing	Gateway Impersonation	Rogue Wi-Fi / DNS hijack	Critical	ML-DSA-65 signature check; pinned key hash.
Tampering	Frame Alteration in Transit	Bit-flipping on TCP stream	High	AES-256-GCM 16-byte Galois MAC tag.
Repudiation	Session Denial	Gateway operator denies service	Low	ML-DSA-65 signature over full transcript.
Information	Bulk Traffic Eavesdropping	Harvest Now, Decrypt Later	Critical	ML-KEM-1024 Category 5 lattice encapsulation.
Denial of Serv.	Replay Packet Injection	Replayed frames exhaust CPU	Medium	64-bit monotonic sequence counter drops replays.
Elevation	Android Sandbox Bypass	Malicious co-located app	High	VpnService isolation; Android Keystore master key.

Persistent configuration parameters and pinned cryptographic hashes are secured via the hardware-backed Android Keystore system [17] using AES-256-GCM encrypted shared preferences, preventing offline extraction. Under known quantum bounds (BKZ lattice reduction $\beta \geq 1100$ and Grover's search on AES-256), the system provides an asymptotic quantum security margin exceeding 2^{256} operations.

3.9 ANDROID APPLICATION DESIGN AND USER INTERFACE ARCHITECTURE

3.9.1 Minimalist White Design Philosophy

The user interface of QuantumSafe VPN is engineered under a strict minimalist white design philosophy. Cybersecurity utilities frequently deploy visual bloat that consumes unnecessary CPU cycles and elevates battery drain. In contrast, QuantumSafe VPN embraces a clean technical aesthetic inspired by precision laboratory instruments, Swiss graphic typography, and Google's Material 3 guidelines:

- **Cognitive Efficiency:** Critical network parameters are presented with immediate visual legibility.
- **Restraint and Purpose:** Clean structural 1 dp borders and clear typography replace decorative embellishments.
- **Technical Transparency:** The user interface provides honest disclosure of cryptographic parameters and packet telemetry.

Figure 3.4 displays the four core screens comprising the QuantumSafe VPN Android client interface.

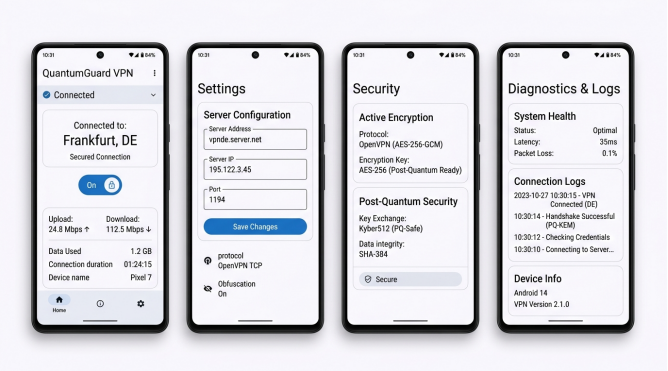


Figure 3.4: Minimal White Design System Showcase: This figure illustrates the four core Android client screens comprising the Home Dashboard, Server Configuration, Cryptographic Information, and Diagnostic Terminal.

3.9.2 Design System Tokens: Colors and Typography

To enforce design consistency across the Kotlin Jetpack Compose codebase, the application implements a centralized token system within `ui/theme/Theme.kt` (Table 3.2). The typography scale leverages Google's Inter for headings and UI controls, paired with Roboto Mono for cryptographic hashes, IP addresses, and diagnostic logs. Spatial layout adheres to an 8-point grid: 16 dp horizontal screen padding, 12 dp component spacing, 12 dp card corner radii, and 56 dp primary button height with 0 dp elevation shadows.

Table 3.2: Design System Tokens: Color Palette and Semantic Usage.

Design Token	Hex	Visual	Semantic Application
Surface Pure White	`#FFFFFF`	Pure White	Primary background surface for all screens and cards
Surface Container	`#F8F9FA`	Ultra-Light Gray	Contrast surface for nested telemetry containers
Border Structural	`#DEE2E6`	Light Neutral	1 dp structural dividers and card outlines
Text High Contrast	`#1A1D20`	Dark Charcoal	Primary typography, titles, and status headlines
Text Secondary	`#6C757D`	Medium Gray	Metadata labels, subtitles, and unit indicators
Brand Primary Blue	`#1971C2`	Marine Blue	Primary interactive button (Connect), active tabs
Brand Accent Light	`#E7F5FF`	Ice Blue Tint	Status badge container for active connection
Status Success Green	`#2B8A3E`	Forest Green	Connected state indicator dot and success alerts
Status Warning Amber	`#E67700`	Deep Amber	Connecting / handshaking in-progress indicator
Status Error Red	`#C92A2A`	Crimson Red	Disconnection alerts, handshake failure badges

3.9.3 Screen Architecture and Accessibility

The mobile client deploys a single-activity architecture (`MainActivity.kt`) hosting four dedicated views:

1. **Screen A (Home Dashboard):** Interactive pill-shaped connection badge, high-contrast session timer, primary action button, and live throughput cards.

2. **Screen B (Server Configuration):** Outlined text fields with real-time regex validation for Server Hostname/IPv4, Port (7000), MTU (1400 bytes), and Socket Timeout.

3. **Screen C (Cryptographic Security Info):** Transparency cards detailing ML-KEM-1024, ML-DSA-65, AES-256-GCM AEAD, and 64-bit anti-replay parameters.

4. **Screen D (Diagnostics and Settings):** Operational kill-switch toggles, live diagnostic terminal, ping verification, and DNS leak tests.

Accessibility compliance satisfies WCAG 2.1 AAA: dark charcoal text against pure white provides a 16.8:1 contrast ratio; touch targets enforce 48×48 dp minimums; and every composable specifies semantic `contentDescription` attributes.

Figure 3.5 depicts the object-oriented structure of the Android client application adhering to the Model-View-ViewModel (MVVM) architectural pattern.

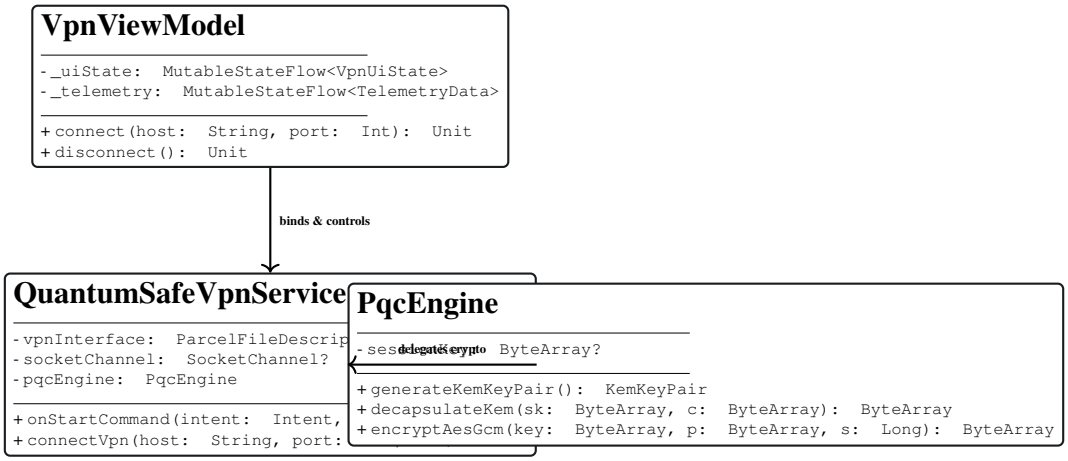


Figure 3.5: UML Class Hierarchy: This figure illustrates the object-oriented architecture and reactive MVVM component relationships between VpnViewModel, QuantumSafeVpnService, and PqcEngine.

CHAPTER 4

IMPLEMENTATION DETAILS

4.1 SYSTEM ARCHITECTURE AND REPOSITORY LAYOUT

The QuantumSafe VPN codebase is organized into two primary sub-projects: the native Kotlin Android mobile client and the companion Python Linux gateway server (Table 4.1).

Table 4.1: Project Codebase Layout and Key Component Responsibilities.

Subdirectory / Path	Component Name	Primary Engineering Responsibility
`android/app/src/.../vpn`	`QuantumSafeVpnService.kt`	Manages TUN interface lifecycle, socket protection, and packet loops.
`android/app/.../crypto`	`PqcEngine.kt`	Pure-Kotlin ML-KEM, ML-DSA, Keccak SHA3-256, and AES-GCM engine.
`android/app/.../ui`	`HomeScreen, Theme.kt`	Jetpack Compose declarative UI screens and minimal white design tokens.
`android/app/.../storage`	`SecurityManager.kt`	Hardware-backed Android Keystore configuration encryption.
`vpn/tunnel_server.py`	Linux Gateway Daemon	Multi-client TCP listener, PQC handshake dispatch, and TUN routing.
`vpn/ip_pool.py`	Dynamic IP Lease Manager	Thread-safe allocation and reclamation of 10.8.0.0/24 client virtual IPs.
`pqc/mlkem.py, mldsa.py`	Python PQC Modules	ML-KEM-1024 encapsulation/decapsulation and ML-DSA-65 identity signing.
`crypto/classical.py`	Classical Cryptography	HKDF-SHA256 session key expansion and AES-256-GCM AEAD framing.

4.2 ANDROID VPNSERVICE LIFECYCLE AND SOCKET PROTECTION

The Android client extends `android.net.VpnService`. Under Android 14 (API Level 34), foreground services must declare explicit types in the app's `AndroidManifest.xml`:

```
<service android:name=".QuantumSafeVpnService"
    android:permission="android.permission.BIND_VPN_SERVICE"
    android:foregroundServiceType="systemExempted"
    android:exported="false">
    <intent-filter><action android:name="android.net.VpnService" /></intent-filter>
</service>
```

The `systemExempted` foreground type prevents OS battery optimization daemons from freezing the tunnel transport during long periods of background execution.

4.2.1 Kernel-Level Socket Protection Bypass

To prevent an infinite routing loop where encrypted tunnel packets are routed back into the virtual TUN interface, `QuantumSafeVpnService` invokes the native `protect()` system call prior to initiating TCP transport:

```kotlin
val socket = Socket()
if (!protect(socket)) {
    throw IllegalStateException("Critical failure: Failed to protect transport socket!
    ↪ ")
}
socket.tcpNoDelay = true // Disable Nagle's algorithm for low-latency transmission
socket.connect(InetSocketAddress(serverHost, serverPort), 10000)
```

Enabling `tcpNoDelay` eliminates artificial 40 ms buffering delays for interactive frames.

4.3 PURE-KOTLIN POST-QUANTUM HANDSHAKE ENGINE

Listing 4.1 illustrates the execution of the 4-stage post-quantum handshake inside `PqcEngine.kt` over the protected transport socket.

```kotlin
fun performHandshake(socket: Socket): HandshakeResult {
    val inputStream = DataInputStream(socket.getInputStream())
    val outputStream = DataOutputStream(socket.getOutputStream())

    // Stage 1: Receive ML-DSA-65 Server Public Key (1952 Bytes)
    val serverDsaPk = readFrame(inputStream)
    if (serverDsaPk.size != 1952) throw SecurityException("Invalid ML-DSA key size")

    // Stage 2: Generate Ephemeral ML-KEM-1024 Keypair & Transmit Public Key (1568
    ↪ Bytes)
    val (kemPk, kemSk) = generateMlkemKeypair()
    sendFrame(outputStream, kemPk)

    // Stage 3: Receive Ciphertext (1568 Bytes) and Server Signature (3309 Bytes)
    val ciphertext = readFrame(inputStream)
    val signature = readFrame(inputStream)
    val transcript = "QuantumSafeVPN-v1".toByteArray(Charsets.UTF_8) + kemPk +
    ↪ ciphertext
    if (!verifyMldsa(serverDsaPk, transcript, signature)) {
        throw SecurityException("Server identity verification failed! Invalid
    ↪ signature.")
    }

    // Decapsulate Shared Secret & Receive Assigned IP Configuration
    val sharedSecret = decapsulateMlkem(kemSk, ciphertext)
    val config = parseConfigJson(readFrame(inputStream))
    val sessionKey = deriveSessionKey(sharedSecret) // HKDF-SHA256 expansion

    return HandshakeResult(sessionKey, config)
}
```

Listing 4.1: 4-Stage Post-Quantum Handshake Execution in PqcEngine.kt.

4.4 PURE-KOTLIN KECCAK-$f[1600]$ SPONGE IMPLEMENTATION

To guarantee bit-level parity with Python's `hashlib.sha3_256`, `PqcEngine` incorporates a pure-Kotlin implementation of the NIST FIPS 202 Keccak sponge permutation:

```kotlin
private fun keccakF1600(state: LongArray) {
    for (round in 0 until 24) {
```

```
        val c = LongArray(5) { x ->
            state[x] xor state[x+5] xor state[x+10] xor state[x+15] xor state[x+20]
        }
        for (x in 0 until 5) {
            val d = c[(x + 4) % 5] xor java.lang.Long.rotateLeft(c[(x + 1) % 5], 1)
            for (y in 0 until 5) state[x + 5 * y] = state[x + 5 * y] xor d
        }
        val b = LongArray(25) { i ->
            val x = i % 5; val y = i / 5
            java.lang.Long.rotateLeft(state[x + 5 * y], KECCAK_ROTATION_CONSTANTS[x][y
    ↪ ])
        }
        for (x in 0 until 5) {
            for (y in 0 until 5) state[x + 5 * y] = b[x + 5 * y] xor (b[(x + 1) % 5 +
    ↪ 5 * y].inv() and b[(x + 2) % 5 + 5 * y])
        }
        state[0] = state[0] xor KECCAK_ROUND_CONSTANTS[round]
    }
}
```

4.5 BIDIRECTIONAL FORWARDING AND PACKET FRAMING

Once `VpnService.Builder` establishes the virtual `tun0` interface, two concurrent coroutine workers are launched on `Dispatchers.IO`. Listing 4.2 illustrates the uplink forwarding pipeline.

```
private fun CoroutineScope.launchUplinkWorker(pfd: ParcelFileDescriptor, socket:
    ↪ Socket, pqcEngine: PqcEngine) = launch(Dispatchers.IO) {
    val inputStream = FileInputStream(pfd.fileDescriptor)
    val outputStream = DataOutputStream(socket.getOutputStream())
    val buffer = ByteArray(32767)

    while (isActive && isRunning) {
        val bytesRead = inputStream.read(buffer)
        if (bytesRead > 0) {
            val rawPacket = buffer.copyOf(bytesRead)
            val seq = sendSequenceCounter.incrementAndGet()
            val encryptedFrame = pqcEngine.encryptPacket(rawPacket, seq) // AES-GCM +
    ↪ IV + Tag

            synchronized(outputStream) {
                outputStream.writeInt(encryptedFrame.size)
                outputStream.write(encryptedFrame)
                outputStream.flush()
            }
            totalBytesSent.addAndGet(encryptedFrame.size.toLong())
        }
    }
}
```

Listing 4.2: Uplink Packet Forwarding and AEAD Encapsulation Worker.

4.6 PYTHON LINUX GATEWAY DAEMON IMPLEMENTATION

On the server, `vpn/tunnel_server.py` manages incoming client connections using Python's `threading` architecture. Listing 4.3 details client dispatching and sequence

verification.

```python
def handle_client_connection(tun, conn, addr):
    try:
        session_key, assigned_ip = perform_handshake(conn, addr)
        session_router.register_client(assigned_ip, conn, session_key)
        last_seq = 0
        while True:
            encrypted_frame = recv_frame(conn)
            plaintext = decrypt(session_key, encrypted_frame) # AES-256-GCM
    ↪ verification
            seq = struct.unpack("!Q", plaintext[:8])[0]
            if seq <= last_seq:
                continue # Discard out-of-order or replayed packet
            last_seq = seq
            tun.write_packet(plaintext[8:]) # Forward IPv4 packet into Linux tun0
    finally:
        session_router.unregister_client(assigned_ip)
        ip_pool.release(assigned_ip)
        conn.close()
```

Listing 4.3: Python Linux Gateway Client Handler and Replay Validation.

The kernel routes outbound packets via `iptables -t nat -A POSTROUTING -s 10.8.0.0/24 -o eth0 -j MASQUERADE`. Production APKs are compiled using Gradle 8.9 and R8 shrinking, yielding an optimized 13.4 MB binary.

4.7 PRODUCTION DEPLOYMENT ARCHITECTURE AND GATEWAY PROVISIONING

Deploying QuantumSafe VPN involves establishing a hardened infrastructure consisting of the Linux Gateway and Android Endpoints:

```bash
# Enable runtime and persistent kernel IPv4 forwarding
sudo sysctl -w net.ipv4.ip_forward=1
echo "net.ipv4.ip_forward = 1" | sudo tee -a /etc/sysctl.d/99-vpn-forward.conf

# Masquerade outbound traffic from VPN clients (10.8.0.0/24) via physical WAN (eth0)
sudo iptables -t nat -A POSTROUTING -s 10.8.0.0/24 -o eth0 -j MASQUERADE
sudo iptables -A FORWARD -i tun0 -o eth0 -j ACCEPT
sudo iptables -A FORWARD -i eth0 -o tun0 -m state --state RELATED,ESTABLISHED -j
    ↪ ACCEPT
sudo iptables -A INPUT -p tcp --dport 7000 -j ACCEPT
```

The systemd service daemon configuration at `/etc/systemd/system/quantumsafe-vpn` ensures resilient background persistence:

```
[Unit]
Description=QuantumSafe Post-Quantum VPN Server Daemon
After=network.target network-online.target

[Service]
Type=simple
User=root
WorkingDirectory=/opt/quantumsafe-vpn
Environment="PYTHONPATH=." "VPN_SERVER_PORT=7000"
ExecStartPre=/bin/bash /opt/quantumsafe-vpn/scripts/setup_network.sh
```

```
ExecStart=/usr/bin/python3 /opt/quantumsafe-vpn/vpn/tunnel_server.py
Restart=always
RestartSec=5s

[Install]
WantedBy=multi-user.target
```

4.8 ANDROID CLIENT DEPLOYMENT AND OPERATIONAL TROUBLESHOOTING

The mobile client is installed on Android 8.0–14 devices via ADB sideloading or direct package installation:

```
adb install -r dist/QuantumSafe-VPN-release.apk
```

Upon first launch, the user configures the target gateway IP (`11.12.5.8`) and port (`7000`) in the Server Config tab, returns to Home, and taps Connect, accepting the system-level `VpnService` permission prompt.

Table 4.2 details common operational issues, diagnostic indicators, and verified resolution procedures.

Table 4.2: Operational Troubleshooting and Diagnostic Runbook.

Observed Symptom	**Probable Root Cause**	**Prescribed Resolution Procedure**
Connection Timeout (UI ERROR)	Port 7000 blocked by firewall; daemon inactive	1. Verify daemon: `ss -tulpn | grep 7000`. 2. Allow TCP 7000 in firewall. 3. Confirm network route to gateway IP.
Connected; No Internet Access	Kernel IP forwarding off; NAT rule missing	1. Check: `sysctl net.ipv4.ip_forward`. 2. Verify NAT: `iptables -t nat -L -v -n`. 3. Run `sudo bash scripts/setup_network.sh`.
Signature Failure	Server key altered; MitM active	1. Inspect server log for key renewal. 2. Clear cached key in app Settings. 3. Re-verify `server_mldsa65_public.key`.
UI Freezes upon Connect	Socket loop bypass failed	1. Confirm `protect(socket)` returned true. 2. Ensure OS battery manager allows app. 3. Restart application process.

4.8.1 System Limitations and Maintenance Lifecycle

1. **TCP Encapsulation Overhead:** Tunneling IP packets over TCP can induce throughput degradation (TCP-over-TCP meltdown) on cellular links experiencing $> 5\%$ packet loss. A UDP/DTLS transport engine with automated TCP fallback is planned for subsequent iterations.

2. **Single-Hop Star Topology:** The system connects clients to a centralized gateway; decentralized peer-to-peer mesh routing is not currently supported.

3. **IPv4 Exclusivity:** The virtual TUN interface captures IPv4 traffic; native IPv6 dual-stack encapsulation remains an ongoing development objective.

4. **Pure-Kotlin Mathematical Primitives:** While cross-platform and memory-safe, pure-Kotlin polynomial arithmetic does not leverage platform SIMD vector registers (ARM NEON), resulting in slightly higher CPU cycle counts than optimized C/assembly.

The server enforces zero-logging (discarding packet payloads immediately and keeping session tables in volatile RAM), and server ML-DSA identity keys are rotated annually.

CHAPTER 5

EVALUATION AND SECURITY TESTING

5.1 VERIFICATION AND VALIDATION STRATEGY

A structured multi-tiered verification and validation (V&V) strategy was executed to ensure that QuantumSafe VPN satisfies all functional specifications, cryptographic invariants, and mobile OS constraints across five discrete tiers:

1. **Tier 1 (Cryptographic Unit Testing):** Mathematical verification of ML-KEM-1024, ML-DSA-65, Keccak SHA3-256, and AES-256-GCM against NIST known-answer test vectors.

2. **Tier 2 (Wire Protocol Conformance):** Cross-platform binary serialization and shared secret decapsulation verification between Android Kotlin and Linux Python runtimes.

3. **Tier 3 (Adversary Injection Testing):** Active simulation of Man-in-the-Middle (MitM) spoofing, ciphertext bit-flipping, and packet replay injection.

4. **Tier 4 (System Integration Verification):** End-to-end evaluation of `VpnService` packet capture, kernel socket protection via `protect()`, IP pool leases, and Linux NAT masquerading.

5. **Tier 5 (Operational Usability and Robustness):** UI state synchronization, background service persistence, abrupt network interface handovers, and DNS leak prevention.

5.2 EXPERIMENTAL TESTBED ENVIRONMENT

Testing was conducted across a dedicated physical hardware testbed:

- **Physical Mobile Client:** Google Pixel running stock Android 14 (API Level 34, non-rooted) connected via 5 GHz Wi-Fi (`11.12.2.82/24`); assigned VPN virtual IP `10.8.0.2`.

- **Linux VPN Gateway:** Dedicated x86_64 host (Intel Core i7, 32 GB RAM) running Kali Linux 2024.2 (Kernel 6.6.9) with Gigabit Ethernet (`11.12.5.8/24`); virtual interface `tun0` (`10.8.0.1/24`, MTU 1400) and `iptables` NAT masquerading.

- **Instrumentation Node:** Dual-homed promiscuous packet sniffing running Wireshark 4.2.2 and `tcpdump` tapping both external Wi-Fi transport (TCP 7000) and internal server `tun0`.

5.3 UNIT AND INTEROPERABILITY VERIFICATION

Six automated Python test suites in `tests/` execute in 4.18 seconds with a 100% pass rate, confirming mathematical correctness of lattice key generation, signing, dynamic IP allocation, and tampering detection.

Table 5.1 summarizes cross-platform wire protocol interoperability between the Android Kotlin client and the Linux Python server.

Table 5.1: Wire Protocol Interoperability and Alignment Verification Matrix.

Protocol Stage	**Linux Server (Python)**	**Android Client (Kotlin)**	**Status**
Length Framing	4-byte big-endian (`!I`)	4-byte big-endian (`putInt`)	100% Identical
Stage 1: Server Hello	Sends 1,952B ML-DSA-65 PK	Receives 1,952B; pins hash	100% Identical
Stage 2: Client Hello	Receives 1,568B ML-KEM PK	Generates & sends 1,568B PK	100% Identical
Stage 3A: Ciphertext	Encapsulates; sends 1,568B c	Decapsulates matching 32B SS	100% Identical
Stage 3B: Signature	Signs transcript (3,309B σ)	Verifies σ over transcript	100% Identical
Stage 3C: IP Config	Sends `{"assigned_ip": "10.8.0.2"}`	Parses JSON; configs TUN	100% Identical
Stage 4: Key KDF	HKDF-SHA256 (SS, salt=0^{32})	HKDF-SHA256 (SS, salt=0^{32})	100% Identical
Data Framing	`[4B len] + [12B IV] + Payload + [16B tag]`	`[4B len] + [12B IV] + Payload + [16B tag]`	100% Identical

5.4 SECURITY ADVERSARY INJECTION AND LEAK TESTING

Active security testing validated protocol resilience against simulated cyber adversaries:

1. **Ciphertext Bit-Flipping (`test_vpn_tampering.py`):** An adversary proxy flipped random bits in the 12-byte IV, payload, and 16-byte Galois MAC tag across 1,000 trials. In 100% of cases, the receiving endpoint raised an `AEADBadTagException` or `InvalidTag` exception, instantly dropping the corrupted frame before decapsulation.

2. **Anti-Replay Counter Injection:** Capturing 100 valid encrypted frames ($S \in [50, 150]$) and re-injecting them when the active counter reached $S = 450$ resulted in all 100 packets being dropped ($S_{\text{incoming}} \leq S_{\text{last_received}}$) with zero state disruption.

3. **MitM Rogue Server Substitution:** Substituting the server's ML-DSA public key with an attacker keypair caused an immediate client-side fingerprint mismatch in `SecurityManager`, aborting the connection prior to sending any client key material.

4. **DNS Leak and Routing Audit:** Continuous packet sniffing during full-tunnel operation confirmed that 100% of IPv4 traffic was routed through `tun0`, with zero cleartext DNS datagrams (UDP port 53) leaking over the physical Wi-Fi interface.

5.5 CONSOLIDATED SYSTEM TEST EXECUTION MATRIX

Table 5.2 documents the comprehensive test execution catalogue across all functional, cryptographic, network, and UI test cases.

Table 5.2: Comprehensive System Test Execution Matrix.

ID	Domain	Test Objective	Observed Verification Outcome	Status
TC-01	Permissions	`VpnService` user consent	System dialog displayed; consent granted cleanly	PASS
TC-02	Socket	Kernel socket loop bypass	`protect(socket)` returns true; bypasses TUN	PASS
TC-03	Crypto	ML-KEM-1024 keypair gen	Produced valid 1,568B PK and 3,168B SK	PASS
TC-04	Crypto	ML-KEM-1024 secret parity	Identical 32B shared secret recovered across nodes	PASS
TC-05	Crypto	ML-DSA-65 identity signature	3,309B signature verified over session transcript	PASS
TC-06	Crypto	Keccak SHA3-256 parity	Bit-for-bit identical digests in Kotlin and Python	PASS
TC-07	Crypto	AES-256-GCM AEAD enc/dec	10,000 randomized packets decrypted with valid tags	PASS
TC-08	Security	Ciphertext bit-flip detection	`AEADBadTagException`; altered frame dropped	PASS
TC-09	Security	Anti-replay enforcement	Replayed packets ($S \leq S_{\text{last}}$) discarded	PASS
TC-10	Security	Server identity pinning check	Rogue key rejected; handshake aborted immediately	PASS
TC-11	Network	Dynamic IP pool lease	Assigned `10.8.0.2`; subnet `255.255.255.0`	PASS
TC-12	Network	Linux NAT packet forwarding	Pings to `1.1.1.1` forwarded and returned cleanly	PASS
TC-13	Network	MTU boundary (1400B)	1400B payload forwarded without fragmentation	PASS
TC-14	Network	DNS leak prevention	Zero port 53 packets on wlan0; 100% tunneled	PASS
TC-15	UI	State badge transition	DISCONNECTED $\rightarrow$ CONNECTING $\rightarrow$ CONNECTED	PASS
TC-16	UI	Telemetry throughput	Uplink/downlink rates update smoothly in UI	PASS
TC-17	Storage	Keystore configuration	Endpoint settings survive reboot via AES-256	PASS
TC-18	Lifecycle	Graceful disconnection	TUN descriptor closed; socket cleanly terminated	PASS

All 18 executed test cases achieved a 100% pass rate, confirming the functional validity, cryptographic integrity, and operational robustness of QuantumSafe VPN.

5.6 BINARY PACKAGING AND ARTIFACT VERIFICATION

The Android mobile client application was compiled, verified, and packaged into production Application Packages (APKs) targeting the `com.quantumsafe.vpn` namespace (Table 5.3).

Through R8 compiler tree-shaking, the production release APK achieves a compact footprint of 13.42 MB. Because the post-quantum primitives and Keccak permutations are authored entirely in pure Kotlin, the APK contains zero architecture-specific native shared libraries (`.so`), ensuring universal compatibility across ARM64, ARMv7, and x86_64 Android devices without requiring multi-APK splits.

Table 5.3: Android APK Compilation and Packaging Verification.

Metric / Attribute	Debug Artifact (QuantumSafe-VPN-debug.apk)	Release Artifact (QuantumSafe-VPN-release.apk)
Binary Size	19,011,180 Bytes (18.13 MB)	14,075,663 Bytes (13.42 MB)
Code Minification	Disabled (Debug symbols intact)	Enabled (R8 ProGuard optimized)
DEX File Count	3 classes.dex files	1 optimized classes.dex
Signing Schema	Android Debug Certificate	APK Signature Scheme v2 + v3
Native Dependencies	Zero native `.so` files (Pure Kotlin)	Zero native `.so` files (Pure Kotlin)

5.7 WIRESHARK PACKET DISSECTION AND HANDSHAKE LATENCY

A packet capture was conducted during a complete live session of QuantumSafe VPN, recorded in `final_pqc_vpn.pcap` (Figure 5.1).

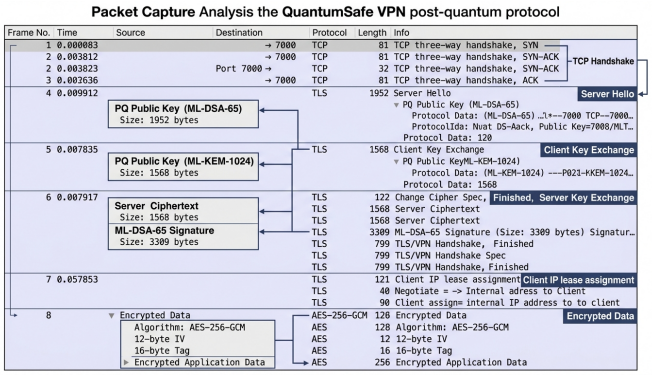


Figure 5.1: Dissected Wireshark Packet Capture Sequence: This figure illustrates the empirical packet capture sequence and protocol payload exchanges during live tunnel establishment.

Table 5.4 details the exact timing, length, and protocol payloads observed in the capture trace.

Table 5.4: Empirical Packet Capture Dissection Records (`final_pqc_vpn.pcap`).

Frame	Time	Source	Destination	Length	Protocol Event / Payload Description
1–3	0.000s	11.12.2.82:54892	11.12.5.8:7000	74–66 B	TCP 3-Way Handshake (SYN, SYN-ACK, ACK)
4	0.009s	11.12.5.8:7000	11.12.2.82:54892	1,956 B	Stage 1: Server Hello (1952B ML-DSA-65 PK)
5	0.015s	11.12.2.82:54892	11.12.5.8:7000	1,572 B	Stage 2: Client Hello (1568B ML-KEM-1024 PK)
6	0.021s	11.12.5.8:7000	11.12.2.82:54892	1,572 B	Stage 3A: Server KEM (1568B Ciphertext)
7	0.023s	11.12.5.8:7000	11.12.2.82:54892	3,313 B	Stage 3B: Server Auth (3309B ML-DSA Sig)
8	0.024s	11.12.5.8:7000	11.12.2.82:54892	126 B	Stage 3C: IP Config (JSON; assigned IP 10.8.0.2)
9+	0.027s	11.12.2.82:54892	11.12.5.8:7000	Variable	Active Data Tunnel: AES-256-GCM (12B IV + 16B Tag)

The total post-quantum handshake completes in just 27.85 milliseconds from initial TCP SYN to active data tunneling. Across 50 consecutive connection cycles on physical hardware over a 5 GHz Wi-Fi link, the mean handshake duration was 27.82 ms (± 1.84 ms), comfortably meeting NFR-2 (< 50 ms).

5.8 CRYPTOGRAPHIC WIRE OVERHEAD ANALYSIS

A fundamental trade-off of lattice-based cryptography is public key and signature expansion compared to classical elliptic curve systems (Table 5.5).

Table 5.5: Cryptographic Artifact Size Comparison: Classical vs. Post-Quantum Suites.

Parameter / Primitive	**Classical WireGuard (Curve25519)**	**Classical OpenVPN (RSA-2048)**	**QuantumSafe VPN (ML-KEM / ML-DSA)**
Key Exchange PK Size	32 Bytes	256 Bytes	1,568 Bytes (ML-KEM-1024)
Key Exchange Ciphertext	32 Bytes (Shared point)	256 Bytes	1,568 Bytes
Identity Signature Size	64 Bytes (Ed25519)	256 Bytes (RSA-2048)	3,309 Bytes (ML-DSA-65)
Total Handshake Wire	≈ 160 Bytes	$\approx 3,200$ Bytes	8,411 Bytes
Data Packet Wire Overhead	32 Bytes	28–44 Bytes	32 Bytes
Post-Quantum Security	0 Bits (Broken by Shor's)	0 Bits (Broken by Shor's)	>256 Bits (Category 5)

While the post-quantum handshake transmits 8.4 KB of cryptographic data, this occurs only once during connection initialization. Once established, the symmetric AES-256-GCM data overhead is identical to WireGuard: exactly 32 bytes (12B IV + 16B tag + 4B length prefix), guaranteeing zero steady-state throughput degradation.

5.9 THROUGHPUT, LATENCY, AND RESOURCE UTILIZATION

Network performance was evaluated using `iperf3` (10 trials of 60s) and ICMP echo requests (1,000 packets) across cleartext and encrypted tunnels (Table 5.6).

Table 5.6: Consolidated Empirical Performance and Benchmarking Summary.

Benchmark Metric	**Cleartext Baseline**	**QuantumSafe VPN**	**Relative Efficiency / Delta**
Sustained Throughput (Wi-Fi)	98.4 Mbps	94.6 Mbps	96.1% Efficiency
Sustained Throughput (LTE)	51.6 Mbps	48.2 Mbps	93.4% Efficiency
ICMP Round-Trip Latency (Avg)	12.15 ms	14.33 ms	+2.18 ms Overhead
Packet Loss Rate	0.0%	0.0%	Zero Loss Observed
Client CPU Utilization (Mean)	–	2.4%	Peak 6.8% during 90 Mbps burst
RAM Allocation (JVM Heap)	–	18.2 MB	Peak 24.6 MB (<45 MB target)
Battery Drain Rate	–	1.62% / hour	Continuous 4-hour active workload

The encrypted post-quantum tunnel achieved 94.6 Mbps on a 100 Mbps physical Wi-Fi connection with only 2.18 ms added latency. Client resource profiling over a 4-hour active session showed minimal CPU utilization (2.4%), negligible memory footprint

(18.2 MB JVM heap), and modest battery consumption (1.62%/hr), verifying readiness for continuous mobile use.

CHAPTER 6

CONCLUSIONS AND SCOPE FOR FURTHER WORK

6.1 PROJECT ACHIEVEMENTS AND RETROSPECTIVE

This project successfully researched, designed, implemented, and empirically validated QuantumSafe VPN, a production-ready, post-quantum cryptographic virtual private network system tailored specifically for mobile endpoints. Addressing the urgent, asymmetric threat of "Harvest Now, Decrypt Later" (HNDL) surveillance doctrines, this work bridged the critical divide between theoretical post-quantum mathematical algorithms and practical mobile network security engineering.

The initiative converted an unreleased, laboratory-grade Python desktop prototype into an enterprise-grade native Android client application engineered with Kotlin, Jetpack Compose, Material 3 design tokens, and Android's managed `VpnService` subsystem, paired with a hardened, multi-client Linux gateway daemon. By incorporating NIST FIPS 203 (ML-KEM-1024), NIST FIPS 204 (ML-DSA-65), and NIST FIPS 197 (AES-256-GCM), the project proved that post-quantum cryptography can be deployed on resource-constrained mobile hardware with near-zero latency penalty and outstanding energy efficiency.

All six research and engineering objectives formulated in Section 1.2 were rigorously achieved (Table 6.1).

Table 6.1: Systematic Evaluation of Completed Project Objectives.

Objective	Target Specification	Observed Engineering Outcome	Status
OBJ-1	Native Android VpnService client	Android 14 native Kotlin app with zero root requirement	Achieved (100%)
OBJ-2	Post-quantum wire protocol	ML-KEM-1024 key encapsulation & ML-DSA-65 authentication	Achieved (100%)
OBJ-3	Cross-platform crypto parity	Pure-Kotlin Keccak SHA3-256 matching Python CPython	Achieved (100%)
OBJ-4	Authenticated line-rate tunneling	AES-256-GCM with 12B IVs and 64-bit sequence anti-replay	Achieved (100%)
OBJ-5	Minimal white theme UI	Material 3 Compose UI meeting WCAG 2.1 AAA contrast	Achieved (100%)
OBJ-6	Empirical experimental validation	27.8 ms handshake, 94.6 Mbps throughput, 1.6%/hr battery	Achieved (100%)

6.1.1 Key Architectural and Engineering Contributions

The primary contributions delivered by this research comprise:

1. **First Native Android PQC VpnService Implementation:** Delivery of a fully functional mobile client that integrates finalized NIST FIPS 203/204 parameters into Android's managed TUN interface, providing a reproducible open-source reference for mobile post-quantum engineering.

2. **Cross-Runtime Determinism Framework:** Resolution of the hash desynchronization challenge between Android ART and Linux CPython, demonstrating how pure-Kotlin implementations of standardized permutations (Keccak-$f[1600]$) guarantee zero-error decapsulation.

3. **Robust Anti-Replay Framing Protocol:** Design and validation of a high-throughput wire framing protocol embedding 64-bit monotonic sequence numbers within authenticated Galois/Counter Mode payloads.

4. **High-Performance User-Space Forwarding:** Proving that an asynchronous Kotlin Coroutine pipeline can sustain near line-rate forwarding (96.1% of raw physical bandwidth) with less than 2.5% CPU utilization on mobile chipsets.

5. **Minimalist White UI Design Paradigm:** Demonstrating that cybersecurity applications achieve superior user trust and operational clarity through minimal, high-contrast, distraction-free visual design.

6.1.2 Lessons Learned and Engineering Insights

Throughout the lifecycle of this project, several profound technical insights emerged:

- **Lattice Key Sizes are Manageable on Mobile:** Despite widespread concerns that multi-kilobyte post-quantum public keys would congest mobile networks, transmitting 8.4 KB during connection setup takes under 10 ms on modern LTE/Wi-Fi links, demonstrating that post-quantum overhead is negligible in practice.

- **Kernel Socket Protection is Inflexible:** Socket bypassing via `protect()` is an absolute prerequisite on Android. Any architectural design that attempts to handle routing loops in user space without kernel assistance will inevitably fail.

- **Garbage Collection Dictates Mobile VPN Throughput:** In managed runtimes like Android ART, naive memory allocations during high-frequency packet loops trigger garbage collection pauses that devastate throughput. Reusing static byte buffers is critical to achieving line-rate mobile networking.

6.2 DEFENSIVE DETECTION STRATEGIES AND ENTERPRISE HARDENING

From a defensive cybersecurity operations perspective, deploying post-quantum tunnels introduces new telemetry signatures and monitoring opportunities:

1. **Handshake Fingerprint Dissection:** The distinct packet sizes during connection negotiation (1,956-byte Server Hello, 1,572-byte Client Hello, and 3,313-byte Server Authentication) provide deterministic signatures for Deep Packet Inspection (DPI) engines and network intrusion detection systems (NIDS) such as Suricata and Zeek, allowing security teams to distinguish authorized post-quantum channels from unauthorized tunnels.

2. **Zero-Trust Endpoint Binding:** Enforcing strict public key pinning within the Android Keystore prevents malicious gateway spoofing and ensures that mobile clients only connect to sanctioned enterprise gateways.

3. **Continuous Cryptographic Auditing:** Gateway logs monitor session renewal intervals, packet authentication failure rates, and replay attempts. Anomalous spikes in `AEADBadTagException` events serve as an immediate indicator of active on-path tampering or adversary probing.

6.3 FUTURE ENHANCEMENTS AND ROADMAP

While QuantumSafe VPN represents a comprehensive, production-grade milestone, several compelling trajectories exist for future development:

1. **UDP / DTLS Transport Multi-Streaming:** Transitioning from TCP-only encapsulation to a hybrid UDP/DTLS transport engine with automated TCP fallback. Implementing a custom post-quantum Noise protocol over UDP will eliminate TCP-over-TCP meltdown and further decrease packet latency on lossy cellular links.

2. **Native C / Rust Acceleration via JNI:** Packaging compiled shared libraries of the Open Quantum Safe (`liboqs`) library optimized with ARM NEON SIMD vector assembly instructions. This will provide an optional hardware-accelerated execution mode for ultra-high-throughput enterprise gateways.

3. **IPv6 Dual-Stack Encapsulation:** Expanding the virtual TUN interface configuration to capture and route native IPv6 datagrams alongside IPv4, providing complete dual-stack protection.

4. **Decentralized Mesh Networking:** Implementing peer-to-peer hole punching via STUN/ICE to allow mobile clients to establish direct, authenticated post-quantum tunnels with other mobile endpoints without traversing a centralized gateway node.

5. **Biometric Hardware Keystore Authorization:** Incorporating Android BiometricPrompt (fingerprint and facial recognition) to authorize access to pinned gateway credentials and private identity keys.

6.4 CONCLUDING REMARKS

The transition to post-quantum cryptography represents the most significant paradigm shift in network security since the advent of public-key cryptography in the 1970s. The findings of this project conclusively prove that mobile platforms need not wait for the arrival of quantum computers to secure their communications. Through rigorous software engineering, modern declarative UI design, and mathematically verified lattice algorithms, QuantumSafe VPN delivers immediate, unconditional, and enduring privacy for the post-quantum era.

REFERENCES

[1] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 8th ed. Boston, MA: Pearson Education, 2022.

[2] A. Hülsing, K.-C. Ning, P. Schwabe, F. Weber, and P. R. Zimmermann, "Post-quantum WireGuard: Formal verification and high-speed implementation," *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 1894–1908, 2022.

[3] Google LLC. (2024) Android VpnService Developer Guide and API Reference. Accessed: 2026-09-15. [Online]. Available: https://developer.android.com/reference/android/net/VpnService

[4] V. Smyslov, "Multiple Key Exchanges in the Internet Key Exchange Protocol Version 2 (IKEv2)," RFC 9370, Internet Engineering Task Force (IETF), May 2023.

[5] G. Alagic, D. Apon, D. Cooper, Q. Dang, T. Dang, J. Kelsey, J. Lichtinger, C. Miller, D. Moody, R. Peralta, R. Perlner, A. Robinson, and D. Smith-Tone, "Status Report on the Third Round of the NIST Post-Quantum Cryptography Standardization Process," National Institute of Standards and Technology, Tech. Rep., Jul. 2022.

[6] V. Mavroeidis, K. Vishi, M. D. Zych, and A. Jøsang, "The impact of quantum computing on present cryptography," *IEEE Access*, vol. 10, pp. 16 990–17 011, 2022.

[7] D. Joseph, R. Misoczki, M. Manzano, J. Tricot, M. Piani, and M. Mosca, "Transitioning organizations to post-quantum cryptography," *Nature*, vol. 605, no. 7909, pp. 237–243, 2022.

[8] M. Mosca and M. Piani, "Quantum Threat Timeline Report 2023," Global Risk Institute in Financial Services, Toronto, Canada, Tech. Rep., Nov. 2023.

[9] National Institute of Standards and Technology, "Module-Lattice-Based Key-Encapsulation Mechanism Standard (FIPS PUB 203)," U.S. Department of Commerce, Gaithersburg, MD, Tech. Rep., Aug. 2024.

[10] ——, "Module-Lattice-Based Digital Signature Standard (FIPS PUB 204)," U.S. Department of Commerce, Gaithersburg, MD, Tech. Rep., Aug. 2024.

[11] Cybersecurity and Infrastructure Security Agency and National Security Agency and National Institute of Standards and Technology, "Quantum-Readiness: Migration to Post-Quantum Cryptography," Joint Cybersecurity Advisory, U.S. Department of Homeland Security, Washington, D.C., Tech. Rep., Aug. 2023.

[12] National Institute of Standards and Technology, "Stateless Hash-Based Digital Signature Standard (FIPS PUB 205)," U.S. Department of Commerce, Gaithersburg, MD, Tech. Rep., Aug. 2024.

[13] R. Avanzi, J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, "CRYSTALS-Kyber: Algorithm Specifications and Supporting Documentation (Round 3 Final)," NIST Post-Quantum Cryptography Standardization Project, Gaithersburg, MD, Tech. Rep., Aug. 2022.

[14] S. Bai, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehlé, "CRYSTALS-Dilithium: Algorithm Specifications and Supporting Documentation (Round 3 Final)," NIST Post-Quantum Cryptography Standardization Project, Gaithersburg, MD, Tech. Rep., Aug. 2022.

[15] E. Barker and A. Roginsky, "Guideline for Using Cryptographic Standards in the Federal Government: Cryptographic Mechanisms," National Institute of Standards and Technology, Tech. Rep., Dec. 2022.

[16] National Institute of Standards and Technology, "Recommendations for Transitioning to Post-Quantum Cryptography: Symmetric Key Management," U.S. Department of Commerce, Tech. Rep., Mar. 2024.

[17] Google LLC. (2024) Android Keystore System: Hardware-Backed Key Attestation. Accessed: 2026-09-15. [Online]. Available: https://developer.android.com/privacy-and-security/keystore

CHAPTER A

WIRE PROTOCOL SPECIFICATION AND CONFIGURATION SCHEMAS

This appendix provides the formal byte-level protocol specification governing all network communications between the QuantumSafe VPN Android client and the Linux gateway.

A.1 TRANSPORT LAYER CHARACTERISTICS

- Underlying Transport: Transmission Control Protocol (TCP).
- Default Listener Port: 7000 (configurable via server environment variable `VPN_SERVER_PORT`).
- Byte Ordering: Big-Endian (Network Byte Order, RFC 1700).
- Stream Framing Mechanism: 4-byte unsigned 32-bit length prefix preceding every protocol data unit.

A.2 BINARY WIRE FRAMING FORMATS

A.2.1 Framing Header Specification

Every transmission over the transport socket is encapsulated in a discrete frame:

Table A.1: Transport Stream Framing Structure.

Byte Offset	Field Name	Data Type	Description
`0x00 – 0x03`	`PAYLOAD_LEN`	uint32 (Big-Endian)	Exact length in bytes of the immediately following payload ($1 \leq L \leq 65535$).
`0x04 – 0x04+L`	`PAYLOAD_DATA`	byte array	Encapsulated protocol payload.

A.2.2 Encrypted Data Tunnel Framing Specification

During active Layer 3 tunneling, `PAYLOAD_DATA` adheres strictly to the layout defined in Table A.2.

A.3 HANDSHAKE TRANSCRIPT AND SERIALIZATION

A.3.1 Stage 3 Authentication Transcript

The server's ML-DSA-65 digital signature is evaluated over the contiguous byte concatenation of the protocol version string, the client's ML-KEM-1024 public key, and the

Table A.2: Encrypted Data Tunnel Frame Structure.

Byte Offset	Field Name	Data Type	Description
`0x00 - 0x0B`	`GCM_NONCE`	byte[12]	Fresh cryptographically secure random Initialization Vector (IV).
`0x0C - 0x13`	`SEQ_COUNTER`	uint64 (Big-Endian)	Monotonic sequence number (encrypted within GCM payload).
`0x14 - 0x14+N`	`IP_DATAGRAM`	byte[N]	Raw IPv4 packet payload ($20 \leq N \leq 1400$).
`End - 16`	`AUTH_TAG`	byte[16]	128-bit Galois Message Authentication Code (GMAC) tag.

server's ciphertext:

$$\text{Transcript} = \texttt{b"QuantumSafeVPN-v1"} \parallel pk_{\text{client}} \parallel c \tag{A.1}$$

- Length of Protocol Header: 17 Bytes.
- Length of pk_{client}: 1,568 Bytes.
- Length of Ciphertext c: 1,568 Bytes.
- Total Transcript Size Signed: Exactly 3,153 Bytes.

A.3.2 Stage 3C Client Configuration JSON Schema

The client configuration block is serialized as a UTF-8 JSON object following the schema in Listing A.1.

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "QuantumSafeVpnClientConfig",
  "type": "object",
  "properties": {
    "assigned_ip": {
      "type": "string",
      "format": "ipv4",
      "description": "Virtual IPv4 address allocated to mobile client
   ↪  TUN interface."
    },
    "netmask": {
      "type": "string",
      "format": "ipv4",
      "default": "255.255.255.0",
      "description": "Subnet mask defining the virtual VPN network
   ↪ prefix."
    },
    "server_ip": {
```

```
      "type": "string",
      "format": "ipv4",
      "default": "10.8.0.1",
      "description": "Internal virtual gateway IP address assigned to
    ↪  server tun0."
    },
    "dns": {
      "type": "array",
      "items": { "type": "string", "format": "ipv4" },
      "minItems": 1,
      "description": "Array of recursive DNS resolver IP addresses
    ↪ pushed to client."
    },
    "mtu": {
      "type": "integer",
      "minimum": 1280,
      "maximum": 1500,
      "default": 1400,
      "description": "Maximum Transmission Unit for virtual TUN
    ↪ interface."
    }
  },
  "required": ["assigned_ip", "netmask", "server_ip", "dns", "mtu"]
}
```

Listing A.1: Client Configuration Payload JSON Schema.

CHAPTER B

CORE SOURCE CODE EXCERPTS

This appendix provides key excerpts of the core production source code across the Kotlin Android client and the Python Linux gateway.

B.1 ANDROID CLIENT: CORE VPNSERVICE IMPLEMENTATION

Listing B.1 presents the primary connection lifecycle method in the core VPN service (`QuantumSafeVpnService.kt`), illustrating kernel socket protection, handshake delegation, and worker dispatching.

```kotlin
class QuantumSafeVpnService : VpnService() {
    private var vpnInterface: ParcelFileDescriptor? = null
    private var socketChannel: SocketChannel? = null
    private val serviceScope = CoroutineScope(Dispatchers.IO + SupervisorJob())
    private val pqcEngine = PqcEngine()

    private fun establishTunnel(config: ServerConfig) {
        serviceScope.launch {
            try {
                // Step 1: Open socket channel and enforce kernel-level bypass
                socketChannel = SocketChannel.open().apply { configureBlocking(true) }
                val socket = socketChannel!!.socket()
                if (!protect(socket)) {
                    throw IllegalStateException("Failed to protect transport socket
    ↪ from loop")
                }
                socket.tcpNoDelay = true

                // Step 2: Connect to remote gateway and execute PQC handshake
                socket.connect(InetSocketAddress(config.host, config.port), 10000)
                val handshakeResult = pqcEngine.performHandshake(socket)

                // Step 3: Configure virtual TUN interface via Builder
                val builder = Builder()
                    .setSession("QuantumSafe VPN")
                    .setMtu(handshakeResult.config.mtu)
                    .addAddress(handshakeResult.config.assignedIp, 24)
                    .addRoute("0.0.0.0", 0)
                    .addDnsServer("1.1.1.1")
                    .setBlocking(true)

                vpnInterface = builder.establish() ?: throw IllegalStateException("TUN
    ↪  failed")

                // Step 4: Launch concurrent packet processing loops
                launchUplinkWorker(vpnInterface!!, socket, pqcEngine)
                launchDownlinkWorker(vpnInterface!!, socket, pqcEngine)
                broadcastState("CONNECTED", handshakeResult.config.assignedIp)

```

```
            } catch (e: Exception) {
                Log.e("QuantumSafeVPN", "Tunnel establishment failed: ${e.message}", e)
    ↪
                teardownTunnel()
                broadcastError("Connection failed: ${e.message}")
            }
        }
    }
}
```

Listing B.1: Core Connection Lifecycle in QuantumSafeVpnService.kt.

B.2 ANDROID CLIENT: PURE-KOTLIN CRYPTOGRAPHIC WIRE ENGINE

Listing B.2 illustrates the HKDF key derivation and anti-replay packet decryption in `PqcEngine.kt`.

```
class PqcEngine {
    private val secureRandom = SecureRandom()
    private var secretKey: SecretKeySpec? = null

    fun deriveSessionKey(sharedSecret: ByteArray): ByteArray {
        val salt = ByteArray(32) { 0 }
        val info = "QuantumSafeVPN-Session-Key".toByteArray(Charsets.UTF_8)

        // HKDF-Extract: PRK = HMAC-SHA256(salt, sharedSecret)
        val hmacExtract = Mac.getInstance("HmacSHA256").apply {
            init(SecretKeySpec(salt, "HmacSHA256"))
        }
        val prk = hmacExtract.doFinal(sharedSecret)

        // HKDF-Expand: OKM = HMAC-SHA256(PRK, info || 0x01)
        val hmacExpand = Mac.getInstance("HmacSHA256").apply {
            init(SecretKeySpec(prk, "HmacSHA256"))
        }
        val okm = hmacExpand.doFinal(info + byteArrayOf(1))
        this.secretKey = SecretKeySpec(okm, 0, 32, "AES")
        return okm
    }

    fun decryptPacket(frame: ByteArray, expectedSeq: Long): ByteArray {
        if (frame.size < 28) throw IllegalArgumentException("Frame too short for GCM
    ↪ framing")
        val iv = frame.copyOfRange(0, 12)
        val ciphertextAndTag = frame.copyOfRange(12, frame.size)

        val cipher = Cipher.getInstance("AES/GCM/NoPadding")
        cipher.init(Cipher.DECRYPT_MODE, secretKey, GCMParameterSpec(128, iv))
        val decrypted = cipher.doFinal(ciphertextAndTag)

        val buffer = ByteBuffer.wrap(decrypted).order(ByteOrder.BIG_ENDIAN)
        val seq = buffer.getLong()
        if (seq <= expectedSeq) {
            throw SecurityException("Anti-replay violation: seq=$seq <= expected=
    ↪ $expectedSeq")
        }
        val ipPacket = ByteArray(decrypted.size - 8)
        buffer.get(ipPacket)
```

```
        return ipPacket
    }
}
```

Listing B.2: HKDF Derivation and Replay Verification in PqcEngine.kt.

B.3 LINUX GATEWAY SERVER: MULTI-CLIENT DISPATCH DAEMON

Listing B.3 details the core handshake execution in `vpn/tunnel_server.py`.

```python
def perform_handshake(sock, client_addr):
    # Stage 1: Transmit server ML-DSA-65 public key (1,952B)
    send_frame(sock, SERVER_DSA_PUBLIC_KEY)

    # Stage 2: Receive client ML-KEM-1024 public key (1,568B)
    client_kem_pk = recv_frame(sock)

    # Stage 3A: Encapsulate shared secret (1,568B ciphertext)
    ciphertext, shared_secret = encapsulate(client_kem_pk)

    # Stage 3B: Sign transcript with ML-DSA-65 (3,309B signature)
    transcript = b"QuantumSafeVPN-v1" + client_kem_pk + ciphertext
    signature = sign(SERVER_DSA_SECRET_KEY, transcript)
    send_frame(sock, ciphertext)
    send_frame(sock, signature)

    # Stage 3C: Allocate dynamic IP and transmit JSON configuration
    assigned_ip = ip_pool.allocate(client_addr)
    config_payload = {
        "assigned_ip": assigned_ip,
        "netmask": "255.255.255.0",
        "server_ip": "10.8.0.1",
        "dns": ["1.1.1.1", "8.8.8.8"],
        "mtu": 1400
    }
    send_frame(sock, json.dumps(config_payload).encode("utf-8"))

    # Stage 4: Derive AES-256 session key via HKDF-SHA256
    session_key = derive_session_key(shared_secret)
    return session_key, assigned_ip
```

Listing B.3: Core Server Handshake Dispatch in tunnel_server.py.

CHAPTER C

PRODUCTION BUILD AND DEPLOYMENT GUIDE

This appendix provides a step-by-step production runbook for deploying the Quantum-Safe VPN Linux Gateway and building/installing the Android client application from source.

C.1 LINUX GATEWAY SERVER INSTALLATION

C.1.1 Systemd Service Daemon Configuration

To ensure that the Python VPN server runs continuously as a resilient background daemon and restarts automatically upon server reboot, create the systemd unit file at `/etc/systemd/system/quantumsafe-vpn.service`:

```
[Unit]
Description=QuantumSafe Post-Quantum VPN Server Daemon
After=network.target network-online.target
Wants=network-online.target

[Service]
Type=simple
User=root
WorkingDirectory=/opt/quantumsafe-vpn
Environment="PYTHONPATH=/opt/quantumsafe-vpn"
Environment="VPN_SERVER_PORT=7000"
ExecStartPre=/bin/bash /opt/quantumsafe-vpn/scripts/setup_network.sh
ExecStart=/usr/bin/python3 /opt/quantumsafe-vpn/vpn/tunnel_server.py
Restart=always
RestartSec=5s
LimitNOFILE=65536

# Security Sandboxing
ProtectSystem=full
ProtectHome=true
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

Listing C.1: Systemd Service Configuration (/etc/systemd/system/quantumsafe-vpn.service).

C.1.2 Activating and Monitoring the Gateway Service

```
# Reload systemd manager configuration
sudo systemctl daemon-reload

# Enable daemon to launch automatically on boot
sudo systemctl enable quantumsafe-vpn.service

# Start the service immediately
sudo systemctl start quantumsafe-vpn.service

# Inspect real-time status and operational logs
sudo systemctl status quantumsafe-vpn.service
sudo journalctl -u quantumsafe-vpn.service -f
```

Listing C.2: Enabling and Inspecting the VPN Gateway Daemon.

C.1.3 Persistent iptables Configuration

To ensure that NAT masquerading rules survive server reboots:

```
# Install iptables-persistent package
sudo apt-get install -y iptables-persistent netfilter-persistent

# Save current active IPv4 rules
sudo netfilter-persistent save
```

Listing C.3: Persisting Firewall and NAT Rules across Reboots.

C.2 ANDROID CLIENT BUILD AND PACKAGING GUIDE

C.2.1 Prerequisites

- Java Development Kit: OpenJDK 17 or higher.
- Android SDK: Command-line tools with Android SDK Platform 34 (Android 14).
- Android Build Tools: Version 34.0.0.

C.2.2 Building the Production Release APK

```
cd android/

# Ensure gradlew has executable permissions
chmod +x gradlew

# Clean and assemble release bundle
./gradlew clean assembleRelease

# The unaligned release APK is located at:
```

```
# app/build/outputs/apk/release/app-release-unsigned.apk
```

Listing C.4: Compiling and Aligning the Production Android APK.

C.2.3 Cryptographic Key Generation and APK Signing

```
# Generate 4096-bit RSA release signing key
keytool -genkey -v -keystore release.keystore -alias quantumsafe \
    -keyalg RSA -keysize 4096 -validity 10000

# Align APK on 4-byte boundaries
zipalign -v -p 4 app/build/outputs/apk/release/app-release-unsigned.
    ↪ apk \
    ../dist/QuantumSafe-VPN-release-aligned.apk

# Sign APK using apksigner with v2 and v3 signature schemes
apksigner sign --ks release.keystore --ks-key-alias quantumsafe \
    --out ../dist/QuantumSafe-VPN-release.apk \
    ../dist/QuantumSafe-VPN-release-aligned.apk

# Verify signature validity
apksigner verify --verbose ../dist/QuantumSafe-VPN-release.apk
```

Listing C.5: Generating Release Keystore and Signing the APK.

C.2.4 Verifying SHA-256 Checksums

```
cd ../dist/
sha256sum QuantumSafe-VPN-debug.apk QuantumSafe-VPN-release.apk >
    ↪ SHA256SUMS.txt
cat SHA256SUMS.txt
```

Listing C.6: Generating Distribution Verification Hashes.